

Desktop UI with Avalonia

POSE 5BHIF, school year 2026/27

Birgit Schröder

2026-09-27

Table of contents

Introduction	3
1 Desktop UI Basics	4
1 Avalonia Basics	5
2 Layouts and Controls	14
3 Styles	22
2 Data Binding and MVVM	29
4 Data Binding and View Models	30
5 Change Notification	32
6 Commands	34
7 The MVVM Pattern	36
8 Lists and Data Templates	39
9 Master/Detail	41
3 Multithreading	43
10 CPU-bound vs. I/O-bound	44
11 Threads and Tasks	47
12 Race Conditions	50
13 Protecting Critical Sections	52
14 The UI Thread	54

Introduction

This book collects the Desktop UI chapters of **POSE** in 5BHIF at HTL Leonding, school year 2026/27. You learn to build desktop applications with **Avalonia** and C#: from your first window to data binding and the MVVM pattern.

The book is meant to be worked through, not just read. Every chapter contains **labs**, highlighted in boxes like this one:

Lab 0.0: How a lab works

 **Goal.** What you will have built at the end of the lab.

The steps: which lines to add and where, and some pieces to write on your own.

 **Check.** What you must see when you run the app. Don't continue until you do.

 **Try it.** An optional experiment.

The labs of a chapter build on each other, so do them in order and keep your project between them.

Tools you need

- the [.NET 10 SDK](#)
- [JetBrains Rider](#) with the *AvaloniaRider* plugin, or [Visual Studio Code](#) with the Avalonia extension
- [Git](#) and a [GitHub](#) account

Part 1

Desktop UI Basics

1 Avalonia Basics

Avalonia is a cross-platform UI framework for .NET. You describe the user interface in **XAML**, an XML-based markup language, and write the logic in C#. The same app runs on Windows, macOS and Linux, and Avalonia can also target the browser and mobile devices.

This chapter covers the foundations: how an Avalonia project is put together, how XAML works, and how a window reacts to clicks. Along the way you build your first app, **HelloAvalonia**, in four labs. Each lab starts from where the previous one stopped, so work through them in order.

1.1 Why Avalonia?

1.1.1 The landscape in 2026

There is no reliable market-share figure for desktop UI frameworks. Most desktop applications are internal tools that never show up in any statistic. The signals that do exist all point the same way:

- **Web frameworks** such as React, Angular and Node.js are used by far more developers than any native desktop toolkit. Many “desktop” apps are web technology too: VS Code, Slack, Teams and Discord are built with Electron.
- **WPF**, Microsoft’s Windows-only desktop framework, has the largest installed base in the .NET world. It is in maintenance mode, but countless enterprise, industrial, medical and finance applications run on it and still have to be maintained and extended.
- **Avalonia** is the fastest-growing .NET desktop framework and the leading cross-platform one. Avalonia 12 was released in April 2026; JetBrains, Unity, Autodesk and NASA are among its users.

As a rough order: web and Electron apps are far ahead, then WPF and WinForms (both legacy), then Avalonia (growing), about level with MAUI and Qt.

1.1.2 Why learn it after three years of web development?

- **A different programming model.** A desktop app is one long-running process that keeps its state, has a UI thread and binds the UI to data. A web app mostly answers stateless requests and re-renders a DOM. Desktop development makes you think about state and threading in ways web development rarely does.
- **WPF comes almost for free.** Avalonia’s XAML, bindings, styles and the MVVM pattern map almost one to one onto WPF. Maintaining existing WPF code is where many junior .NET jobs are: machine control panels, test benches, in-house tools.
- **It runs everywhere**, including on your own laptop, whatever operating system it has.
- **Some software doesn’t belong in a browser:** anything that talks to hardware (serial ports, USB, fieldbuses), runs offline or in isolated networks, computes heavily on the local machine, or drives an embedded Linux touch panel.
- **Same stack as the backend.** It is the same C#, the same tooling, the same dependency injection and the same unit tests you already know from ASP.NET Core.
- **The patterns transfer.** Observable state, commands and composing a UI from components are close relatives of what React and Angular do.

To be honest about it: most of you will build more web frontends than desktop UIs. Avalonia widens your skill set; it doesn’t replace your web skills.

1.2 Setting up

You need:

- the **.NET 10 SDK**,
- an IDE: **JetBrains Rider** with the *AvaloniaRider* plugin, or **Visual Studio Code** with the *Avalonia for VS Code* extension,

- the Avalonia project templates. Install them once per machine:

```
dotnet new install Avalonia.Templates
```

Lab 1.1: Create and run the project

Goal. An empty Avalonia app that starts and shows a window.

1. Create the project and a solution file for it:

```
dotnet new avalonia.app -n HelloAvalonia -o HelloAvalonia
cd HelloAvalonia
dotnet new sln -n HelloAvalonia
dotnet sln add HelloAvalonia.csproj
```

The solution file (`HelloAvalonia.slnx`) is optional for a single project, but it lets Rider open the folder as a solution.

2. Open `HelloAvalonia.csproj` and add one line to the first `<PropertyGroup>`:

```
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>
```

From now on every compiler warning stops the build. Warnings are almost always real problems (a possible `null`, an unused variable), and this way you can't ignore them.

3. Start the app:

```
dotnet run
```

Check. A window titled *HelloAvalonia* opens and shows the text *Welcome to Avalonia!*. Close it before you continue.

1.3 The project files

An Avalonia application strictly separates **what the UI looks like** from **what it does**. The layout lives in a XAML file with the extension `.axaml`; the logic lives in a `C#` file of the same name with `.axaml.cs` appended, the **code-behind**. Your new project contains:

File	Purpose
<code>Program.cs</code>	The Main entry point. Configures and starts the Avalonia application.
<code>App.axaml</code> / <code>App.axaml.cs</code>	The application object: the global theme, and which window opens at startup.
<code>MainWindow.axaml</code>	The view : the layout and controls of the main window, in XAML.
<code>MainWindow.axaml.cs</code>	The code-behind : the <code>C#</code> logic that belongs to <code>MainWindow.axaml</code> .
<code>HelloAvalonia.csproj</code>	The project file: target framework and NuGet packages.
<code>app.manifest</code>	Windows-only settings. Leave it as it is.

Rider shows each `.axaml` file with its `.axaml.cs` file nested underneath, because the two belong together.

The extension is `.axaml`, not `.xaml`, although the language is XAML. The different extension only exists so that IDEs never confuse Avalonia files with those of Microsoft's XAML frameworks, which use `.xaml`.

1.3.1 How the application starts

`Program.Main` builds the application from the `App` class and starts it. `App.axaml` defines the application-wide resources, most importantly the theme:

```
<Application xmlns="https://github.com/avaloniaui"
              xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
              x:Class="HelloAvalonia.App"
              RequestedThemeVariant="Default">
  <Application.Styles>
    <FluentTheme />
  </Application.Styles>
</Application>
```

```

    </Application.Styles>
</Application>

```

`RequestedThemeVariant="Default"` follows the operating system's light or dark mode. Which window opens first is decided in code, in `App.axaml.cs`:

```

public override void OnFrameworkInitializationCompleted()
{
    if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime desktop)
    {
        desktop.MainWindow = new MainWindow();
    }

    base.OnFrameworkInitializationCompleted();
}

```

`Program.cs` also contains a call to `.WithDeveloperTools()`, surrounded by `#if DEBUG`. It connects debug builds to Avalonia's developer tools; we don't need them in this chapter, so leave the line as it is.

1.3.2 How XAML and code-behind are connected

Look at the top of `MainWindow.axaml`:

```

<Window ...
    x:Class="HelloAvalonia.MainWindow">

```

`x:Class` tells the XAML compiler that this markup belongs to the C# class `HelloAvalonia.MainWindow` (namespace plus class name). In `MainWindow.axaml.cs` that class is declared **partial**:

```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}

```

At build time, Avalonia generates the **second half** of this partial class from the XAML file: a field for every named control, and the code that builds the controls. `InitializeComponent()`, called once in the constructor, runs that generated code. Without it the window would stay empty. Two files, one class.

The XAML is compiled straight into the program; it is not parsed at runtime. A typo in a property name is therefore a **build error**, not a crash when the window opens.

1.4 XAML basics

XAML uses XML syntax, so the usual XML rules apply:

- A file has exactly one **root element**, and all other elements are correctly nested inside it. For a window, the root is `<Window>`; for a reusable part of a UI, it is a `<UserControl>`.
- Element and attribute names are **case-sensitive**, and attribute values are always in double quotes.
- The nesting of elements forms a **tree**. Avalonia turns that tree into the UI.

Every element stands for a class. When the window is created, `<Button>` creates a `Button` object through its parameterless constructor. Every attribute sets a public **property** of that object or subscribes to one of its **events**:

```

<Button Background="AliceBlue" Content="Click me!"/>

```

The attribute values are strings. Behind the scenes, **type converters** turn them into the right type, for example `"AliceBlue"` into a brush.

1.4.1 Namespaces

The root element of every XAML file declares at least two XML namespaces:

```
xmlns="https://github.com/avaloniaui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

- The first is the **default namespace** (no prefix). It contains all Avalonia controls, so you can write `<Button>` directly.
- The second maps the XAML language namespace to the prefix `x`. Language features such as `x:Class` and `x:Name` come from there.

To use your own classes in XAML, declare a namespace for them with a prefix of your choice:

```
xmlns:models="using:HelloAvalonia.Models"
```

Then `<models:Person/>` creates a `HelloAvalonia.Models.Person`. For a class in a different assembly (another project or a NuGet package), the older form names the assembly as well: `xmlns:logic="clr-namespace:LogicNamespace;assembly=Logic"`.

1.4.2 Property element syntax

Some values are too complex for a string. Instead of an attribute, you can then write the property as a **child element** named `<Type.Property>`:

```
<Button Background="AliceBlue">
  <Button.Content>
    <Ellipse Width="120" Height="90" Fill="LightBlue"/>
  </Button.Content>
</Button>
```

Here the button's `Content` is no longer text but an `Ellipse`. `Content` is also the button's *content property*, so you may leave out `<Button.Content>` and put the `Ellipse` directly between `<Button>` and `</Button>`.

1.4.3 Naming elements

To access an element from code, give it a name:

```
<TextBlock Name="CounterText" Text="Counter: 0"/>
```

The generated half of the partial class then contains a field `CounterText` of type `TextBlock`: the exact object the XAML created. `x:Name="CounterText"` does the same. `Name` is a property of the Avalonia controls, while `x:Name` is a XAML language feature that works on any element. For controls, use `Name`.

1.4.4 Markup extensions

Sometimes a property should not get a fixed value but one that comes from somewhere else. That is the job of **markup extensions**, written in curly braces. Here, the second button takes its background from the first:

```
<Button Name="Btn1" Background="LightYellow"/>
<Button Background="{Binding #Btn1.Background}"/>
```

`#Btn1` refers to the element named `Btn1`. The most important markup extensions are:

Extension	Description
Binding	Takes the value from a data source and keeps it up to date (chapter <i>Data Binding and View Models</i>).
StaticResource	Takes the value from a resource, once, when the element is created.
DynamicResource	Takes the value from a resource and follows it when the resource changes.
TemplateBinding	Used inside a control template (chapter <i>Styles</i>).

1.4.5 Styled properties

Avalonia controls don't use plain C# properties for most of their settings. They use **styled properties**, which are registered with Avalonia's property system (`AvaloniaProperty.Register`). You set and read them like any other property, but they can do more:

- they have default values,

- they notify Avalonia whenever their value changes,
- they can validate new values and run a callback when the value changes.

That is what makes data binding, styles and animations possible: all three only work with styled properties. Every property of a built-in control that you would want to style or bind already is one.

1.4.6 Attached properties

An **attached property** is set on one element but belongs to another, usually the layout container around it. In this example, each button says at which edge of the `DockPanel` it wants to be docked:

```
<DockPanel>
  <Button DockPanel.Dock="Top">Click me!</Button>
  <Button DockPanel.Dock="Bottom">Click me too!</Button>
</DockPanel>
```

`DockPanel.Dock` only makes sense on a child of a `DockPanel`. Without attached properties, `Button` would need a `Dock` property, plus a property for every other container it could ever be placed in. The same mechanism places elements in the rows and columns of a grid, as you'll see next.

1.5 A first layout

A `Window` can only have **one** direct child. To show more than one element, that child is a **layout container** (a *panel*), and everything else goes inside it. The two most common panels are:

- **Grid** divides the space into rows and columns. `RowDefinitions="*,Auto"` creates two rows: the second is exactly as tall as its content (`Auto`), and the first takes all the remaining space (`*`). Children choose their row with the attached property `Grid.Row`.
- **StackPanel** arranges its children one after another, vertically by default or horizontally with `Orientation="Horizontal"`. `Spacing` adds a gap between the children.

The chapter *Layouts and Controls* covers all panels in detail.

Lab 1.2: The layout

Goal. The window shows a greeting, a counter and four buttons. The buttons don't do anything yet. All changes are in `MainWindow.axaml`.

1. In the `<Window>` start tag, change the title to `Title="Hello Avalonia"` and add after it:

```
Width="480" Height="360" MinWidth="420" MinHeight="320"
WindowStartupLocation="CenterScreen"
```

Also set `d:DesignWidth="480"` and `d:DesignHeight="360"`. The `d:` attributes only apply to the previewer in the IDE; `mc:Ignorable="d"` tells the compiler to skip them.

2. Replace the text `Welcome to Avalonia!` with a grid of two rows:

```
<Grid RowDefinitions="*,Auto" Margin="32">
</Grid>
```

3. Inside the grid, add the upper row, a `StackPanel` with the greeting:

```
<StackPanel Grid.Row="0" Spacing="20"
  VerticalAlignment="Center" HorizontalAlignment="Center">
  <TextBlock Text="Hello, Avalonia!" FontSize="36" FontWeight="SemiBold"
    HorizontalAlignment="Center"/>
</StackPanel>
```

4. In that `StackPanel`, below the greeting, add a button and the counter:

```
<Button Content="Click me!" Classes="accent" Padding="24,10"
        HorizontalAlignment="Center"/>
<TextBlock Text="Counter: 0" FontSize="16" Opacity="0.7"
        HorizontalAlignment="Center"/>
```

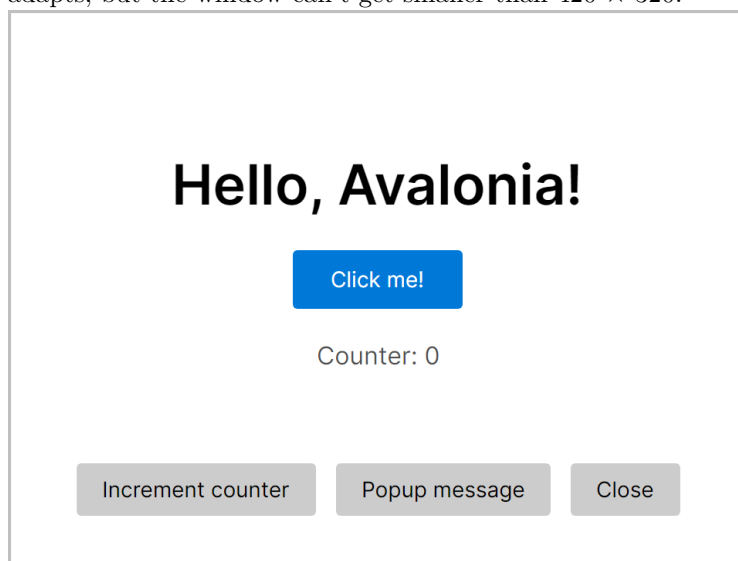
`Classes="accent"` paints the button in the system's accent color (chapter *Styles*).

5. Below that `StackPanel`, still inside the grid, add the lower row:

```
<StackPanel Grid.Row="1" Orientation="Horizontal" Spacing="12"
            HorizontalAlignment="Center">
    <Button Content="Increment counter" Padding="16,8"/>
</StackPanel>
```

Then add two more buttons to it yourself: *Popup message* and *Close*, with the same padding.

✓ **Check.** `dotnet run`: the window is centered on the screen, the greeting and counter sit in the middle, and the three buttons are lined up along the bottom. Make the window narrower and wider: the layout adapts, but the window can't get smaller than 420×320 .



1.6 Events

To react to user input, you attach a **handler method** to an **event**. The method runs every time the event occurs. There are two ways to attach it.

In XAML, name the method in the event attribute. The method must exist in the code-behind:

```
<Button Name="DemoButton" Content="Click me!" Click="OnDemoButtonClicked"/>

private void OnDemoButtonClicked(object? sender, RoutedEventArgs e)
{
    DemoButton.Content = "Clicked!";
}
```

In code, subscribe to the event in the constructor, after `InitializeComponent()` has created the controls:

```
public MainWindow()
{
    InitializeComponent();
    DemoButton.Click += OnDemoButtonClicked;
}
```

Both do the same. The labs use the XAML form. Later, with MVVM, most event handlers are replaced by *commands*.

A few conventions that the handlers in this book follow:

- Handlers are **private**: only the window itself needs them.

- The signature is (object? sender, RoutedEventArgs e). sender is the control that raised the event.
- Names follow the pattern On<Control><Event>, e.g. OnCloseButtonClicked, so you can quickly find the handler of a XAML attribute.

1.6.1 Routed events

Click is a **routed event**: it doesn't stay at the button you clicked. It first fires on the button, then on the button's parent, then on *its* parent, all the way up to the window. This is called **bubbling**. Because of it, one handler on a container can react to clicks on any button inside it.

When two nested controls both handle the same event, the inner handler can stop it with `e.Handled = true`, and the outer control never sees it.

1.6.2 State lives in the code-behind

As long as there is no data binding, the state of the window is just a field in the `MainWindow` class. A handler changes the field and then updates the UI **by hand**:

```
private int _counter;

private void OnIncrementCounterButtonClicked(object? sender, RoutedEventArgs e)
{
    _counter++;
    CounterText.Text = "Counter: " + _counter;
}
```

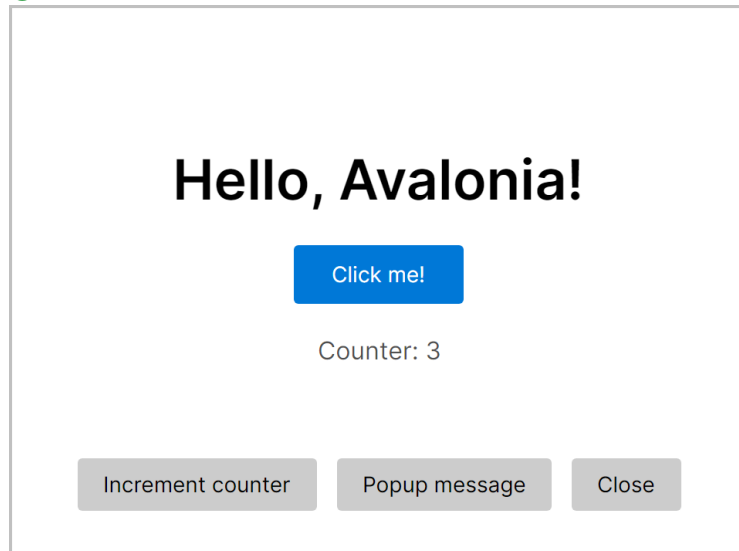
Forgetting the second line is a classic bug: the state changes, the screen doesn't. Data binding (chapter *Data Binding and View Models*) removes exactly this manual step.

Lab 1.3: The counter

🎯 **Goal.** *Increment counter* counts up, and *Close* closes the window.

1. In `MainWindow.axaml`, add `Name="CounterText"` to the counter's `TextBlock`, and `Click="OnIncrementCounterButtonClicked"` to the *Increment counter* button.
2. In `MainWindow.axaml.cs`, add `using Avalonia.Interactivity;` at the top (for `RoutedEventArgs`). Then add the field `_counter` and the method `OnIncrementCounterButtonClicked` from the section *State lives in the code-behind* to the class.
3. Now the *Close* button, on your own: add `Click="OnCloseButtonClicked"` to it and write the handler. A window closes itself with `Close()`.

✅ **Check.** Each click on *Increment counter* raises the number by one. *Close* ends the app.



💡 **Try it.** Rename `OnCloseButtonClicked` in the C# file only and build. The XAML compiler checks that every handler named in XAML exists, but its message is not very clear: *AVLN3000: Unable to find suitable setter or adder for property Click*. It does point at the right line and column of `MainWindow.axaml`, though.

Remember this message: it almost always means a misspelled handler name. Undo the rename afterwards.

1.7 Dialogs

Avalonia itself has no ready-made message box. The NuGet package **MessageBox.Avalonia** provides one. It can show a dialog in two ways:

- **ShowAsync()** opens it as a separate window.
- **ShowAsPopupAsync(this)** shows it as an overlay inside **this** window.

Both methods return a **Task**, so they are awaited, and the handlers that call them are marked **async**. A handler is then **async void**, which is normally a mistake in C#: nobody can await the method, and an exception thrown in it can't be caught by the caller. **Event handlers are the one accepted exception**, because events require methods that return void. Every other **async** method, including those called from an event handler, returns a **Task**.

Lab 1.4: Dialogs

🎯 **Goal.** *Click me!* asks whether to quit, and *Popup message* shows a message inside the window.

1. Add the package:

```
dotnet add package MessageBox.Avalonia --version 12.0.0
```

2. In `MainWindow.axaml`, add `Click="OnGreetButtonClicked"` to the *Click me!* button.
3. In `MainWindow.axaml.cs`, add two lines at the top:

```
using MsBox.Avalonia;  
using MsBox.Avalonia.Enums;
```

and this handler to the class:

```
private async void OnGreetButtonClicked(object? sender, RoutedEventArgs e)  
{  
    var box = MessageBoxManager.GetMessageBoxStandard(  
        "Hello", "Hello from Avalonia! Do you want to quit?", ButtonEnum.YesNo);  
    var result = await box.ShowAsync();  
    if (result == ButtonResult.Yes)  
    {  
        Close();  
    }  
}
```

4. Now *Popup message*, on your own: attach a handler `OnShowPopupButtonClicked` and show a message box with only a title and a text (no `ButtonEnum`). Show it with `ShowAsPopupAsync(this)` instead of `ShowAsync()`.

✅ **Check.** *Click me!* opens a separate dialog window; *Yes* closes the app, *No* only the dialog. *Popup message* shows the message as an overlay inside the main window, not as a window of its own.

1.8 Window events

Besides the events of its controls, a window has events of its own. The most useful ones:

Event	Occurs when
Opened	the window has been shown for the first time.
Loaded	the window and all its controls are created and laid out. Initialization code that needs the controls belongs here.
Activated / Deactivated	the window becomes the active window, or another window does.
PositionChanged	the window was moved.
SizeChanged	the window's size changed.

Event	Occurs when
Closing	the window is about to close. Setting <code>e.Cancel = true</code> in the handler keeps it open, e.g. to ask about unsaved changes.
Closed	the window has been closed.

There is no event for minimizing or maximizing. The `WindowState` property holds that state; subscribe to `PropertyChanged` and check whether `e.Property` is `Window.WindowStateProperty` to be notified.

1.9 The previewer

Rider (with the *AvaloniaRider* plugin) and VS Code (with the Avalonia extension) render a live preview of an `.axaml` file next to the editor, without starting the app. That's handy for adjusting a layout. The `d:` attributes from Lab 1.2 set the size of that preview.

The previewer is **not completely reliable**. It can lag behind your edits, keep showing an error you already fixed, or fail to load for reasons that have nothing to do with your XAML. A rebuild usually clears it. A red previewer is no proof that your XAML is wrong: check with `dotnet build` first.

1.10 Try it yourself

Extend `HelloAvalonia`:

1. Add a button that resets the counter to 0.
2. Disable *Increment counter* once the counter reaches 10. Every control has an `IsEnabled` property.
3. Swap `ShowAsync` and `ShowAsPopupAsync` in the two dialog handlers and compare.
4. Add a `TextBox` and change the greeting dialog to greet the name entered there. A `TextBox` has a `Text` property.
5. Handle the window's `Closing` event: ask *"Do you really want to quit?"* and keep the window open when the answer is *No*. Watch out: the handler can't wait for the dialog before deciding, because the window closes as soon as the handler returns. Cancel first, ask, and call `Close()` yourself on *Yes*. Then make sure that this second `Close()` doesn't ask again.

2 Layouts and Controls

A window can hold only one child. Everything else is arranged by **panels**: containers that decide where their children go and how big they are. This chapter introduces the panels you will use all the time, plus a few helpers around them.

In the labs you build **LayoutDemo**: a launcher window with one button per panel, each opening a small window where you try that panel out.

2.1 Layout basics

- Panels (classes derived from `Panel`) hold any number of children. All other controls hold at most one child, usually in their `Content` property.
- Panels can be nested: a `Grid` cell can contain a `StackPanel`, which contains another `Grid`, and so on. Real layouts are almost always nested.
- Avoid fixed sizes. Let the panels size the controls, so the layout adapts when the window is resized or the text changes.
- Layout runs in two passes: in the **measure** pass every element reports how much space it would like; in the **arrange** pass the panel assigns each child its final position and size.

Every control also has a few properties that tell its panel how to place it:

Property	Meaning
<code>HorizontalAlignment</code> , <code>VerticalAlignment</code>	Stretch (fill the available space), Left/Top , Center , Right/Bottom
<code>Margin</code>	Space around the control: "10" on all sides, "10,5" left/right and top/bottom, "10,5,10,0" left, top, right, bottom
<code>Padding</code>	Space inside the control, between its border and its content
<code>Width</code> , <code>Height</code> , <code>MinWidth</code> , <code>MaxWidth</code> , ...	Fixed or limited sizes; use sparingly

Lab 2.1: The launcher

🎯 **Goal.** A launcher window with a *Grid* button that opens a second window.

1. Create the project as in Lab 1.1, but named `LayoutDemo`, including the solution file and `TreatWarningsAsErrors`.
2. In `MainWindow.axaml`, set `Title="Layout Demo"` `Width="360"` `Height="560"` and replace the text `Welcome to Avalonia!` with:

```
<StackPanel Margin="24" Spacing="10">
  <TextBlock Text="Layouts and Controls" FontSize="22" FontWeight="SemiBold"/>
  <Button Content="Grid" HorizontalAlignment="Stretch" Click="OnOpenGridClicked"/>
</StackPanel>
```

3. Create a second window. In the project folder, run:

```
dotnet new avalonia.window -n GridWindow
```

This creates `GridWindow.axaml` and `GridWindow.axaml.cs`. In `GridWindow.axaml`, set `Title="Grid"` `Width="620"` `Height="560"`.

4. In `MainWindow.axaml.cs`, add `using Avalonia.Interactivity`; and this handler:

```
private void OnOpenGridClicked(object? sender, RoutedEventArgs e) => new
    GridWindow().Show();
```

`Show()` opens the window next to the launcher; you can open it several times. (Its sibling

ShowDialog(this) would block the launcher until the new window is closed.)

✓ **Check.** The launcher shows a heading and a full-width *Grid* button. Each click opens a window *Grid* with the text *Welcome to Avalonia!*.

All further labs of this chapter follow the same pattern: create a window with `dotnet new avalonia.window`, give it a title and a size, and add a launcher button with a handler.

2.2 Grid

A *Grid* arranges its children in rows and columns. Each child picks its cell with the attached properties `Grid.Row` and `Grid.Column` (both start at 0, which is also the default). Several children may share a cell; the one declared later is drawn on top.

```
<Grid ShowGridLines="True">
  <Grid.RowDefinitions>
    <RowDefinition/>
    <RowDefinition/>
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition/>
    <ColumnDefinition/>
    <ColumnDefinition/>
  </Grid.ColumnDefinitions>
  <Button Grid.Row="0" Grid.Column="1" HorizontalAlignment="Center">Button 1</Button>
  <Ellipse Grid.Row="1" Grid.Column="2" Fill="Beige"/>
  <Button Grid.Row="1" Grid.Column="2" HorizontalAlignment="Center"
    Background="Transparent">Button 2</Button>
</Grid>
```

This grid has two rows and three columns. *Button 2* sits on top of the beige ellipse in the same cell. `ShowGridLines` draws the cell borders; it is a debugging aid, remove it when the layout is done.

The row and column definitions can also be written as a compact attribute:

```
<Grid RowDefinitions="*,*" ColumnDefinitions="*,*,*">
```

2.2.1 Sizing rows and columns

Every row height and column width is one of three kinds:

- a **fixed** size in pixels: `Width="100"`,
- **Auto**: as big as the content needs,
- **star** (*): a share of the remaining space. * and 2* split it 1 : 2. A definition without a size counts as *.

```
<Grid ColumnDefinitions="100,Auto,*,2*">
```

The first column is always 100 pixels wide, the second fits its content, and the third and fourth share the rest in the ratio 1 : 2, growing and shrinking with the window.

2.2.2 Spanning cells

`Grid.RowSpan` and `Grid.ColumnSpan` let a child cover several rows or columns, like `rowspan/colspan` in an HTML table:

```
<Label Grid.Row="4" Grid.ColumnSpan="2" Content="Rectangles showing column width"/>
```

Lab 2.2: Grid basics

🎯 **Goal.** The grid example from above, in the first tab of the *Grid* window.

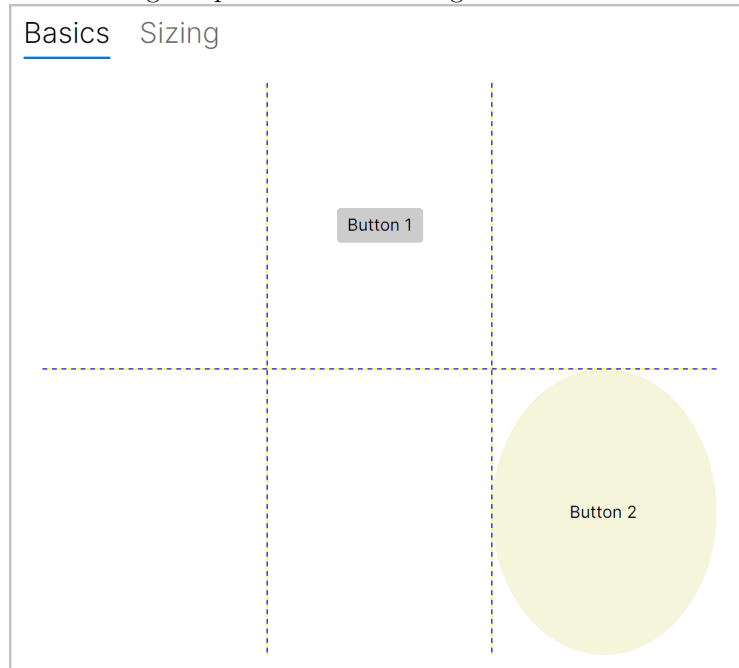
1. In `GridWindow.axaml`, replace the text `Welcome to Avalonia!` with a `TabControl` with one tab:

```
<TabControl>
  <TabItem Header="Basics">
  </TabItem>
</TabControl>
```

A `TabControl` shows one `TabItem` at a time; the headers switch between them.

2. Put the grid example from the section *Grid* into the tab, and give the grid a `Margin="16"`.

✓ **Check.** The *Basics* tab shows the dotted grid lines, *Button 1* in the middle of the top row, and *Button 2* over the beige ellipse at the bottom right.



💡 **Try it.** Replace the two definition blocks by the compact attributes. Then remove `HorizontalAlignment="Center"` from *Button 1*.

Lab 2.3: Sizing rows and columns

🎯 **Goal.** A second tab that shows the three kinds of sizes side by side.

1. Add a second tab to the `TabControl` in `GridWindow.axaml`:

```
<TabItem Header="Sizing">
  <Grid Margin="16" RowDefinitions="*,*,2*,2*,*,Auto" ColumnDefinitions="Auto,*">
    </Grid>
  </TabItem>
```

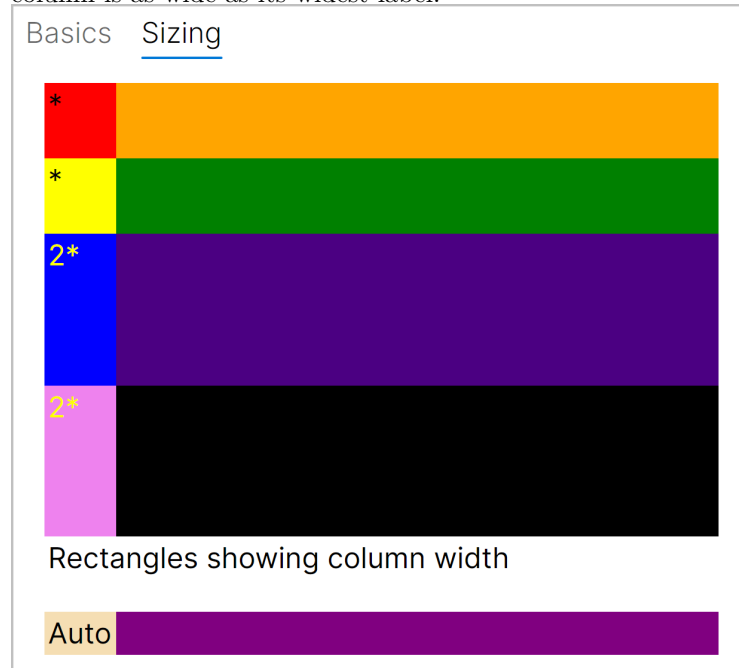
2. Fill row 0 of the grid:

```
<Rectangle Grid.Row="0" Fill="Red"/>
<Label Grid.Row="0" Content="*" FontSize="24"/>
<Rectangle Grid.Row="0" Grid.Column="1" Fill="Orange"/>
```

3. Fill the other rows yourself, following this table. Where the label is on a dark color, give it `Foreground="Yellow"`.

Row	Left column (Auto)	Label	Right column (*)
1	Yellow	*	Green
2	Blue	2*	Indigo
3	Violet	2*	Black
4	—	<i>Rectangles showing column width, spanning both columns</i>	
5	Wheat	Auto	Purple

✓ **Check.** Rows 2 and 3 are twice as tall as rows 0, 1 and 4; row 5 is exactly as tall as its label. The left column is as wide as its widest label.



💡 **Try it.** Make the window taller and shorter: which rows change their height, and which don't?

2.3 StackPanel

A `StackPanel` puts its children one after another, vertically by default or horizontally with `Orientation="Horizontal"`. `Spacing` adds a gap between them.

```
<StackPanel Background="Aquamarine">
  <TextBlock Text="First name"/>
  <TextBox/>
  <TextBlock Text="Last name"/>
  <TextBox Width="100" HorizontalAlignment="Left"/>
</StackPanel>
```

In a vertical `StackPanel` every child is as wide as the panel, unless it sets its own width and alignment like the second `TextBox`.

Lab 2.4: StackPanel

🎯 **Goal.** A `StackPanel` window with the example above.

1. Create `StackPanelWindow` (title *StackPanel*, 420×380) and its launcher button, as in Lab 2.1.
2. Replace the window's text with the `StackPanel` example, and give the panel a `Margin="16"`.

✓ **Check.** A turquoise block with the two labels and input fields, the second field only 100 pixels wide.

💡 **Try it.** Set `Orientation="Horizontal"`. What happens to the first `TextBox`, which has no width of its own? Add `Spacing="8"`.

2.4 DockPanel

A `DockPanel` attaches each child to one of its edges with the attached property `DockPanel.Dock` (`Top`, `Bottom`, `Left`, `Right`). The **order matters**: each child is docked into the space the previous children left over. With `LastChildFill="True"` (the default), the last child fills whatever remains.

In the direction it is docked, a child only gets the size it asks for. A button docked to the top is therefore only as tall as its text, even with `VerticalAlignment="Stretch"`; give it a `Height` to make it taller (a `Width` for `Left/Right`). Along the edge, `Stretch` fills the full length.

Lab 2.5: DockPanel

🎯 **Goal.** Five buttons docked to different edges.

1. Create `DockPanelWindow` (title *DockPanel*, 420×320) and its launcher button.

2. Replace the window's text with a `DockPanel` and its first two buttons:

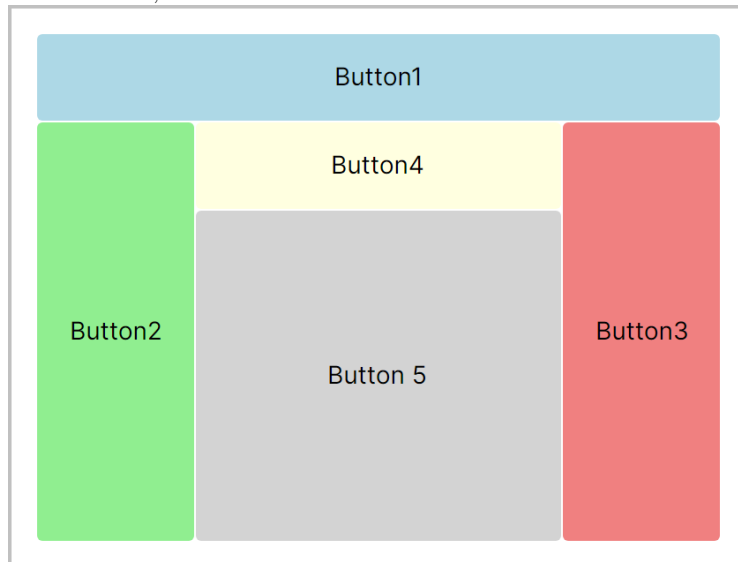
```
<DockPanel Margin="16">
  <Button DockPanel.Dock="Top" Background="LightBlue" Height="50"
    HorizontalAlignment="Stretch"
    HorizontalContentAlignment="Center"
    ↪ VerticalContentAlignment="Center">Button1</Button>
  <Button DockPanel.Dock="Left" Background="LightGreen" Width="90"
    VerticalAlignment="Stretch"
    HorizontalContentAlignment="Center"
    ↪ VerticalContentAlignment="Center">Button2</Button>
</DockPanel>
```

`HorizontalAlignment/VerticalContentAlignment` center the text inside the larger button.

3. Add three more buttons yourself:

- *Button3*, docked `Right`, `LightCoral`, 90 wide,
- *Button4*, docked `Top`, `LightYellow`, 50 tall,
- *Button 5*, not docked, `LightGray`, stretched in both directions.

✔ **Check.** *Button1* spans the full width at the top. *Button4* is also at the top, but only between *Button2* and *Button3*, because it came later. *Button 5* fills the middle.



💡 **Try it.** Move *Button1* below *Button2* in the XAML. Then set `LastChildFill="False"`.

2.5 WrapPanel

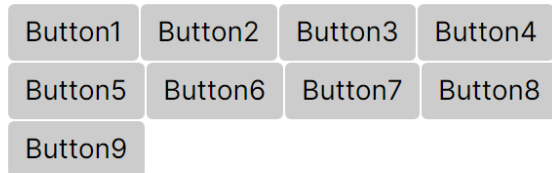
A `WrapPanel` works like a horizontal `StackPanel`, but starts a new line when the next child doesn't fit any more. A `StackPanel` would simply let it run off the edge. With `Orientation="Vertical"` it fills columns instead of lines.

Lab 2.6: WrapPanel

🎯 **Goal.** Nine buttons that wrap when the window gets narrow.

1. Create `WrapPanelWindow` (title *WrapPanel*, 380×360) and its launcher button.
2. Replace the window's text with a `<WrapPanel Margin="16">` containing nine buttons, *Button1* to *Button9*.

✔ **Check.** The buttons fill a line, then continue on the next one. Make the window narrower and wider: the lines re-flow.



💡 **Try it.** Set `Orientation="Vertical"` and resize the window again.

2.6 Canvas

A `Canvas` doesn't arrange anything: you position every child yourself, with the attached properties `Canvas.Left`, `Canvas.Top`, `Canvas.Right` and `Canvas.Bottom`. It is the only panel that works with coordinates. That makes it right for drawings and diagrams, and wrong for forms: nothing adapts when the window or the text size changes.

```
<Canvas>
  <TextBlock Canvas.Top="10" Canvas.Left="10" Text="First name"/>
  <TextBox Canvas.Top="30" Canvas.Left="10" Width="200"/>
</Canvas>
```

Lab 2.7: Canvas

🎯 **Goal.** A small form, positioned by coordinates.

1. Create `CanvasWindow` (title *Canvas*, 360×300) and its launcher button.
2. Replace the window's text with the `Canvas` example above; add `Margin="16"` and `Background="AliceBlue"` to the canvas.
3. Add a *Last name* label and input field yourself, 50 pixels below the first pair.

✅ **Check.** Both pairs are shown on a light blue area, the input fields 200 pixels wide.

💡 **Try it.** Make the window very narrow: the input fields are cut off instead of getting smaller.

2.7 UniformGrid

A `UniformGrid` is a grid whose cells all have the same size. You don't place the children; they fill the cells in order. `Rows` and `Columns` fix the number of rows or columns; the other number follows from the number of children.

Lab 2.8: UniformGrid

🎯 **Goal.** Six buttons of equal size in two rows.

1. Create `UniformGridWindow` (title *UniformGrid*, 360×320) and its launcher button.
2. Replace the window's text with:

```
<UniformGrid Margin="16" Rows="2">
  <Button HorizontalAlignment="Stretch" VerticalAlignment="Stretch"
    HorizontalContentAlignment="Center"
    ↵ VerticalContentAlignment="Center">Button1</Button>
</UniformGrid>
```

3. Add *Button2* to *Button6* the same way.

✓ **Check.** Two rows of three buttons, all of the same size, filling the window.

💡 **Try it.** Replace `Rows="2"` by `Columns="2"`, then remove one button.

2.8 GridSplitter

A `GridSplitter` is a bar in a `Grid` that the user can drag to change the width of columns (or the height of rows) at runtime. Two properties matter:

- **ResizeDirection:** always set it explicitly, to `Columns` or `Rows`.
- **HorizontalAlignment** (for columns) decides *which* columns change: `Left` changes the splitter's column and the one before it, `Right` its column and the one after it, and `Stretch` (the default) the columns on both sides of it. A splitter in the first column therefore needs `HorizontalAlignment="Right"`; without it, it tries to resize a column before column 0, and dragging does nothing.

`MinWidth/MaxWidth` on a `ColumnDefinition` limit how far the user can drag.

Lab 2.9: GridSplitter

🎯 **Goal.** A grid whose column width can be changed with the mouse.

1. Create `GridSplitterWindow` (title *GridSplitter*, 420×360) and its launcher button.
2. Replace the window's text with a `<Grid Margin="16">` of two rows and two columns. Put `<Button Content="Button 1" Margin="10"/>` into the top-left cell and *Button 2* into the bottom-right cell.
3. Add the splitter at the end of the grid:

```
<GridSplitter Grid.Column="0" Grid.RowSpan="2" Width="9" Background="Gray"
  ResizeDirection="Columns" HorizontalAlignment="Right"/>
```

✓ **Check.** A gray bar runs down the right edge of the left column. Dragging it makes the left column wider or narrower.

💡 **Try it.** Remove `HorizontalAlignment="Right"` and drag again. Then add `MinWidth="80"` to the first column definition (you need the long form of the definitions for that).

2.9 ScrollViewer

Panels never scroll. If the content can be larger than the space available, wrap it in a `ScrollViewer`. It shows a vertical scrollbar when needed; the horizontal one is off unless you set `HorizontalScrollBarVisibility` (`Auto` shows it when needed, `Visible` always).

Two ways to break a line in a text:

- in an attribute value, the XML character reference `
`: `Text="a
b"`,
- inside a `TextBlock`'s content, the element `<LineBreak/>`.

One more thing about panels: a panel only receives mouse events where it has a background. If a panel should react to clicks but not be colored, set `Background="Transparent"`.

Lab 2.10: ScrollViewer

🎯 **Goal.** Content that is wider than the window, with a scrollbar.

1. Create `ScrollViewerWindow` (title *ScrollViewer*, 420×340) and its launcher button.
2. Replace the window's text with:

```
<ScrollView Margin="16" HorizontalScrollBarVisibility="Visible">
    <Grid MinWidth="600" ColumnDefinitions="*,*">
    </Grid>
</ScrollView>
```

3. In the grid's first column, add a `TextBox` with `Margin="10"` and the text *a* and *b* on two lines. In the second column, add a `TextBlock` with `Margin="10"` whose content is `a b c d e f g, a`
`<LineBreak/>`, and `h i`.

✔ **Check.** A horizontal scrollbar is shown below the content. Scroll right to see the second column.

💡 **Try it.** Set `HorizontalScrollBarVisibility="Auto"` and make the window wider than 600 pixels.

2.10 Building a UI in code

Everything you write in XAML can also be written in C#: XAML elements are objects, attributes are properties. Attached properties are set with static methods such as `DockPanel.SetDock(button, Dock.Top)`.

```
var dPanel = new DockPanel();
var btnTop = new Button();
var btnLeft = new Button();

btnTop.Content = "Top Button!";
btnTop.Height = 50;
btnTop.HorizontalAlignment = HorizontalAlignment.Stretch;
btnTop.HorizontalContentAlignment = HorizontalAlignment.Center;
btnTop.VerticalContentAlignment = VerticalAlignment.Center;

btnLeft.Content = "Left Button!";
btnLeft.Width = 95;
btnLeft.VerticalAlignment = VerticalAlignment.Stretch;
btnLeft.HorizontalContentAlignment = HorizontalAlignment.Center;
btnLeft.VerticalContentAlignment = VerticalAlignment.Center;

DockPanel.SetDock(btnTop, Dock.Top);
DockPanel.SetDock(btnLeft, Dock.Left);

dPanel.LastChildFill = false;
dPanel.Children.Add(btnTop);
dPanel.Children.Add(btnLeft);

Content = dPanel;
```

Keep this for the rare cases where the UI is only known at runtime, for example one button per entry of a list read from a file. For everything else XAML is shorter, easier to read and shows up in the previewer.

Lab 2.11: A window built in code

🎯 **Goal.** A window whose content is created in C# only.

1. Create `CodeBehindWindow` (title *Code-Behind*, 360×260) and its launcher button.
2. In `CodeBehindWindow.axaml`, delete the text `Welcome to Avalonia!`, so the window has no content.
3. In `CodeBehindWindow.axaml.cs`, add `using Avalonia.Layout;` (for the alignments), and add the code from the section above to the constructor, after `InitializeComponent()`.

✔ **Check.** A button *Top Button!* across the top and *Left Button!* on the left; the rest of the window stays empty because `LastChildFill` is `false`.

💡 **Try it.** Add a third button in code, without a dock, and set `LastChildFill` to `true`.

2.11 Try it yourself

1. Build a login form with a `Grid`: two rows with a label and an input field each, labels in an `Auto` column, fields in a `*` column, and a right-aligned *Login* button below.
2. Rebuild the *DockPanel* window with a `Grid`. Which layout needs less XAML?
3. The launcher of `LayoutDemo` is too long for a small window. Make it scroll.

3 Styles

Setting `FontSize="20"` on every button of an app is tedious and error-prone. A **style** sets properties in one place, for all controls it applies to. Avalonia styles work much like CSS: a **selector** picks the controls, **setters** set their properties.

In the labs you build **StylesDemo**: one window with a tab per topic.

If you search the web for help, you will often find WPF answers. WPF styles work differently (keys, `TargetType`, triggers); in Avalonia, look for *selectors*.

3.1 Styles and selectors

A style is a `Style` element with a `Selector` and one or more `Setters`:

```
<Style Selector="Button">
  <Setter Property="FontStyle" Value="Italic"/>
  <Setter Property="FontSize" Value="20"/>
</Style>
```

The selector `Button` matches **every** button in the style's scope. To style only some buttons, give them a **style class** and add it to the selector with a dot:

```
<Style Selector="Button.bigText">
  <Setter Property="FontSize" Value="20"/>
</Style>
```

```
<Button Classes="bigText" Content="Create new employee"/>
```

An element can have several classes (`Classes="bigText danger"`), and code can add and remove them at runtime (`myButton.Classes.Add("bigText")`). The selector always needs a type in front of the class (`Button.bigText`, not `.bigText`), so that the compiler knows which `FontSize` the setter means.

Other useful selector parts:

Selector	Matches
<code>Button</code>	every <code>Button</code>
<code>Button.bigText</code>	every <code>Button</code> with the class <code>bigText</code>
<code>#SaveButton</code>	the element named <code>SaveButton</code>
<code>StackPanel Button</code>	every <code>Button</code> somewhere inside a <code>StackPanel</code>
<code>StackPanel > Button</code>	every <code>Button</code> that is a direct child of a <code>StackPanel</code>

3.1.1 Where styles live

Styles are collected in the `Styles` property of a control, and apply to that control and everything inside it:

- `<StackPanel.Styles>` (or any other control): only inside that panel,
- `<Window.Styles>`: in this window,
- `<Application.Styles>` in `App.axaml`: in the whole application. The `<FluentTheme/>` already there is itself a large collection of styles.

Styles you want to share between several windows can also go into a file of their own and be included with `<StyleInclude Source="/Styles/Buttons.axaml"/>`.

Lab 3.1: Style classes

🎯 **Goal.** A window with a first tab, where a style class makes two of three buttons bigger.

1. Create the project as in Lab 1.1, named `StylesDemo`.
2. In `MainWindow.axaml`, set `Title="Styles Demo"` `Width="640"` `Height="420"` and replace the text `Welcome to Avalonia!` with:

```

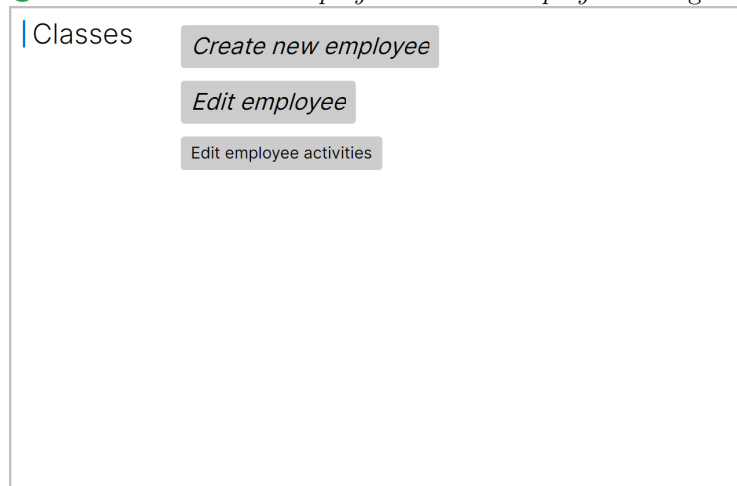
<TabControl TabStripPlacement="Left">
  <TabItem Header="Classes">
    <StackPanel Margin="16" Spacing="10">
      <StackPanel.Styles>
        <Style Selector="Button.bigText">
          <Setter Property="FontStyle" Value="Italic"/>
          <Setter Property="FontSize" Value="20"/>
        </Style>
      </StackPanel.Styles>
      <Button Content="Create new employee"/>
    </StackPanel>
  </TabItem>
</TabControl>

```

TabStripPlacement="Left" puts the tab headers on the left, so there is room for many tabs.

3. Add two more buttons below the first: *Edit employee* and *Edit employee activities*. Then give the first two buttons the class `bigText`.

✓ **Check.** *Create new employee* and *Edit employee* are big and italic, *Edit employee activities* is not.



💡 **Try it.** Change the selector to `Button` and watch which buttons change. Then give a `TextBlock` the class `bigText`.

3.2 Cascading

When several styles match an element, **all** of them apply: their setters are merged. If two of them set the same property:

- a style whose selector contains a class or pseudo-class beats one with only a type (`Button.special` beats `Button`, wherever they are written),
- otherwise the style written **later** wins. There is no “more specific wins” rule as in CSS: `TextBlock.caption.small` only beats `TextBlock.caption` if it comes after it.

That makes variants easy. Instead of copying a style, add a second, more specific one that only changes what is different:

```

<Style Selector="TextBlock.caption">
  <Setter Property="FontSize" Value="30"/>
  <Setter Property="FontWeight" Value="Bold"/>
  <Setter Property="Margin" Value="0,0,0,20"/>
</Style>
<Style Selector="TextBlock.caption.small">
  <Setter Property="FontSize" Value="20"/>
</Style>

<TextBlock Classes="caption" Text="A form"/>
<TextBlock Classes="caption small" Text="A smaller form"/>

```

The second `TextBlock` matches both styles. It gets the bold weight and the margin from `caption`, and the font

size from `caption.small`, which comes later.

Lab 3.2: Cascading

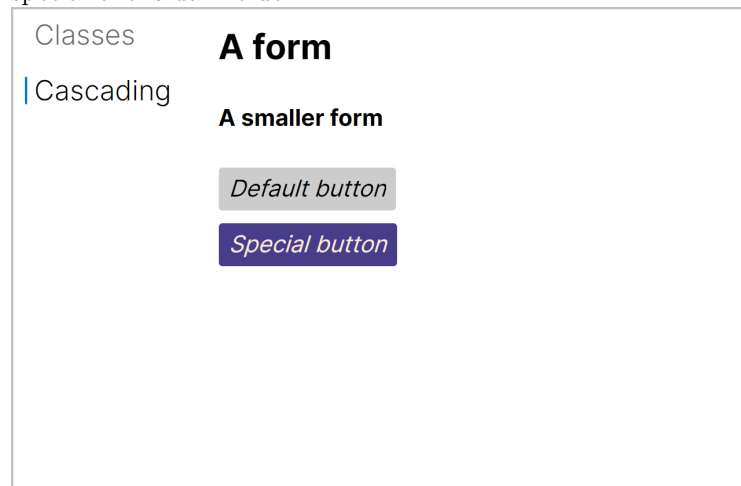
🎯 **Goal.** A caption and a smaller variant of it, plus a special button that builds on the normal button style.

1. Add a second tab below the first one:

```
<TabItem Header="Cascading">
  <StackPanel Margin="16" Spacing="10">
    <StackPanel.Styles>
    </StackPanel.Styles>
  </StackPanel>
</TabItem>
```

2. Add the two caption styles and the two `TextBlocks` from the section above to the tab.
3. Add two buttons, *Default button* and *Special button* (with `Classes="special"`), and write two styles for them yourself:
 - every `Button` in this tab: italic, font size 20,
 - buttons with the class `special`: additionally `Foreground AntiqueWhite` and `Background DarkSlateBlue`.

✅ **Check.** *A smaller form* is bold like *A form*, but smaller. Both buttons are italic and big; only the special one is dark blue.



💡 **Try it.** Move the `caption.small` style above the `caption` style. Then move the `Button.special` style above the `Button` style.

3.3 Pseudo-classes

Controls change their look in certain states: a button is hovered or pressed, a text box has the keyboard focus. Selectors react to such states with **pseudo-classes**, written with a colon:

Pseudo-class	The control ...
<code>:pointerover</code>	is under the mouse pointer
<code>:pressed</code>	is being pressed
<code>:focus</code>	has the keyboard focus
<code>:disabled</code>	has <code>IsEnabled="False"</code>
<code>:checked</code>	is checked (<code>CheckBox</code> , <code>ToggleButton</code> , ...)

```
<Style Selector="Button:pointerover">
  <Setter Property="FontWeight" Value="Bold"/>
</Style>
```

The style applies while the state lasts, and its setters are undone automatically when the state ends. Conditions are combined by writing them one after the other, without spaces: `Button.save:pointerover` matches only a

button with the class `save` that is hovered.

3.4 Transitions and animations

Style changes happen instantly. To animate them, give the property a **transition**: from then on, every change of the property's value is animated, in both directions.

```
<Style Selector="TextBox">
  <Setter Property="FontSize" Value="12"/>
  <Setter Property="Transitions">
    <Transitions>
      <DoubleTransition Property="FontSize" Duration="0:0:0.2"/>
    </Transitions>
  </Setter>
</Style>
<Style Selector="TextBox:focus">
  <Setter Property="FontSize" Value="20"/>
</Style>
```

The text box grows when it gets the focus and shrinks again when it loses it. There are transitions for other property types, too: `ColorTransition`, `BrushTransition`, `ThicknessTransition`, `TransformOperationsTransition` and more.

For animations with several steps, a style can contain **keyframe animations**. They run when the style starts to apply:

```
<Style Selector="Button.fade:pointerover">
  <Style.Animations>
    <Animation Duration="0:0:0.2" FillMode="Forward">
      <KeyFrame Cue="0%">
        <Setter Property="Opacity" Value="1"/>
      </KeyFrame>
      <KeyFrame Cue="100%">
        <Setter Property="Opacity" Value="0.4"/>
      </KeyFrame>
    </Animation>
  </Style.Animations>
</Style>
```

`FillMode="Forward"` keeps the value of the last keyframe after the animation has finished.

Lab 3.3: Pseudo-classes

🎯 **Goal.** A *Save* button that turns bold on hover, and a *Cancel* button that turns italic while pressed.

1. Add a third tab `States`, built like the *Cascading* tab: a `StackPanel` with `Margin="16"` `Spacing="10"` and empty `StackPanel.Styles`.
2. Add two buttons: `<Button Classes="save" Content="Save"/>` and `<Button Classes="cancel" Content="Cancel"/>`.
3. Write the two styles yourself: `Button.save:pointerover` sets `FontWeight` to `Bold`, and `Button.cancel:pressed` sets `FontStyle` to `Italic`.

✅ **Check.** *Save* is bold only while the mouse is over it. *Cancel* turns italic only while you hold the mouse button down.

💡 **Try it.** Remove `.save` from the selector: now *Cancel* turns bold on hover, too.

Lab 3.4: Transitions

🎯 **Goal.** A text box that grows smoothly when it gets the focus.

1. Add a fourth tab `Animation`, built like the others.
2. Add the `TextBox` style with the transition from the section above to the tab's styles, and two text boxes: `<TextBox Text="Focus me"/>` and `<TextBox Text="Or me"/>`.
3. Write the `TextBox:focus` style yourself, with font size 20.

✅ **Check.** Click into a text box: it grows smoothly. Click into the other: the first shrinks back while the second grows.

💡 **Try it.** Add the keyframe example from the section above and a `<Button Classes="fade" Content="Hover to fade"/>`. Then remove `FillMode="Forward"`.

3.5 Classes from data

A style class can be switched on and off by a binding. The syntax is `Classes.<name>`, bound to a `bool`:

```
<CheckBox Name="WarningCheck" Content="Show as warning"/>
<TextBlock Classes.warning="{Binding #WarningCheck.IsChecked}" Text="Disk almost full"/>
```

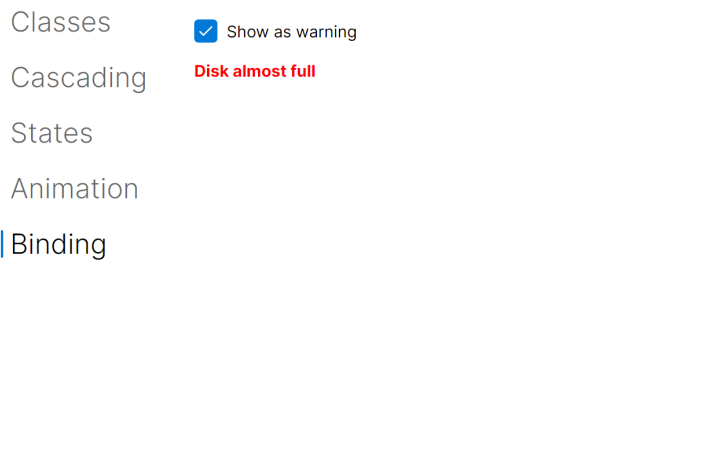
While the check box is checked, the `TextBlock` has the class `warning`, and a style `TextBlock.warning` applies. This is how a style reacts to data: the chapter *Master/Detail* uses the same pattern to highlight list entries, bound to a property of the data instead of a check box.

Lab 3.5: Classes from data

🎯 **Goal.** A check box that turns a message red.

1. Add a fifth tab `Binding`, built like the others.
2. Add the `CheckBox` and the `TextBlock` from the section above.
3. Write the style `TextBlock.warning` yourself: `Foreground Red`, `FontWeight Bold`.

✅ **Check.** Checking the box turns the text red and bold; unchecking it turns it back.



3.6 Resources

A **resource** is a named object, stored in the `Resources` of a control or the application and used with `{StaticResource Key}`. Colors are the typical example: define them once, use them everywhere.

```
<Application.Resources>
  <SolidColorBrush x:Key="BrandBrush" Color="DarkSlateBlue"/>
</Application.Resources>

<Setter Property="Foreground" Value="{StaticResource BrandBrush}"/>
```

`StaticResource` looks the resource up once. `DynamicResource` keeps watching it, which is needed when the resource can change at runtime, for example when switching between light and dark theme.

Lab 3.6: An app-wide style

🎯 **Goal.** Every tab gets a title, styled once for the whole application.

1. In `App.axaml`, add the `BrandBrush` resource from above, and below `<FluentTheme />` a style for titles:

```
<Style Selector="TextBlock.title">
  <Setter Property="FontSize" Value="22"/>
  <Setter Property="FontWeight" Value="SemiBold"/>
  <Setter Property="Foreground" Value="{StaticResource BrandBrush}"/>
</Style>
```

2. At the top of each tab's `StackPanel`, add a `TextBlock` with `Classes="title"` and the tab's name as text.

✅ **Check.** All five tabs have a dark blue title, although the style is written only once.

💡 Try it. Change the color of `BrandBrush` and see all titles change.

3.7 Control templates

A style can only set properties that a control has. To change what a control is **made of**, you give it a new **control template**: the tree of elements that draws the control. A template for a button could be a colored `Border` with a `TextBlock` inside:

```
<ControlTemplate x:Key="ButtonTemplate" TargetType="Button">
    <Border Name="border" BorderBrush="Blue" BorderThickness="3" CornerRadius="3"
        Background="Azure" Padding="3" Margin="3">
        <TextBlock Margin="{TemplateBinding Padding}" Text="{TemplateBinding Content}"
            VerticalAlignment="Center" HorizontalAlignment="Center"/>
    </Border>
</ControlTemplate>
```

```
<Button Template="{StaticResource ButtonTemplate}" Content="Templated button"/>
```

`{TemplateBinding Content}` takes the value from the button the template is applied to, so the same template works for every button text.

To style an element **inside** a template, the selector uses `/template/`:

```
<Style Selector="Button:pointerover /template/ Border#border">
    <Setter Property="Background" Value="LightBlue"/>
</Style>
```

This reads: the `Border` named `border` inside the template of a hovered `Button`.

3.7.1 Styling the parts of the built-in templates

The Fluent theme's own templates have named parts, too. That matters because some states are drawn by the template, not by the button itself. A style `Button:pointerover` that sets `Background` has **no visible effect**: while hovered, the theme paints the template part `PART_ContentPresenter` with its own color. To change the hover color, style that part:

```
<Style Selector="Button.primary">
    <Setter Property="Background" Value="DarkSlateBlue"/>
    <Setter Property="Foreground" Value="White"/>

    <Style Selector="^:pointerover /template/ ContentPresenter#PART_ContentPresenter">
        <Setter Property="Background" Value="MediumPurple"/>
    </Style>
</Style>
```

The inner style is **nested**: `^` stands for the selector of the outer style, so the inner selector means `Button.primary:pointerover /template/ ContentPresenter#PART_ContentPresenter`.

To replace the look of a control type everywhere, Avalonia uses a `ControlTheme` instead of styles. That goes beyond this chapter.

Lab 3.7: Control templates

🎯 **Goal.** A button with a template of its own, and a *primary* button with its own hover color.

1. In `MainWindow.axaml`, add a `<Window.Resources>` block before the `TabControl` and put the `ButtonTemplate` from the section above into it.
2. Add a sixth tab `Templates`, with a title as in Lab 3.6, and the button `<Button Template="{StaticResource ButtonTemplate}" Content="Templated button"/>`.
3. Add the `/template/` style for the hovered border to the tab's styles.
4. Add a button with `Classes="primary"` and the text *Primary button*, and the nested `Button.primary` style from the section above.

✅ **Check.** The templated button is a light blue box with a blue frame; it turns a darker blue on hover. The primary button is dark blue with white text and turns purple on hover.

Classes **Templates**

Cascading

Templated button

States

Primary button

Animation

Binding

| Templates

💡 **Try it.** Replace the nested hover style by `Button.primary:pointerover` with a `Background` setter, and hover again.

3.8 Try it yourself

1. Give all buttons of the *States* tab a `Transitions` block, so that their `Opacity` changes smoothly, and a `:pointerover` style that sets `Opacity` to 0.7.
2. Move the two caption styles from the *Cascading* tab into a file `Styles/Captions.axaml` and include it in `App.axaml` with `StyleInclude`.
3. Write a control template for a `CheckBox` that shows *ON* or *OFF* instead of a check mark. Hint: the pseudo-class `:checked` works in `/template/` selectors, too.

Part 2

Data Binding and MVVM

4 Data Binding and View Models

In HelloAvalonia, every change of the state needed a line like `CounterText.Text = "Counter: " + _counter;`. Forget it once, and the screen shows old data. **Data binding** connects a control property to a data property, and keeps the two in sync automatically. On top of it sits the **MVVM** pattern, which this course uses from now on.

This part has two lab projects. In the chapters up to *The MVVM Pattern*, you rebuild HelloAvalonia with MVVM as **HelloMvvm**. From *Lists and Data Templates* on, you build **DatabindingDemo**, a list of people with a detail form.

4.1 Data binding

A binding connects a **target**, a property of a control, with a **source**, a property of some object. The target must be a styled property (chapter *Avalonia Basics*); every property of the built-in controls that you would want to bind is one.

You have met bindings already, with another element as the source:

```
<TextBox Name="TBRight"/>
<TextBox Text="{Binding #TBRight.Text}"/>
```

4.1.1 Binding modes

Mode	Direction
OneWay	source → target: the control shows the value
TwoWay	source ↔ target: the control shows the value, and user input changes it
OneTime	source → target, once, when the binding is created
OneWayToSource	target → source only

You rarely set a mode yourself. Properties that users edit (`TextBox.Text`, `CheckBox.IsChecked`, `Slider.Value`, `ListBox.SelectedItem`, ...) default to `TwoWay`, all others to `OneWay`. User input is written to the source immediately, on every keystroke.

4.2 View models and the DataContext

MVVM (Model-View-ViewModel) splits an application into three parts:

- the **View**: the window, its XAML and a minimal code-behind,
- the **ViewModel**: a plain C# class holding the state of the view and the actions on it, as properties and commands. It knows nothing about windows or controls,
- the **Model**: the data and business logic behind it.

The view finds its view model through the **DataContext**. Set it once, usually in the window's constructor:

```
public MainWindow()
{
    InitializeComponent();
    DataContext = new MainWindowViewModel();
}
```

The `DataContext` is inherited by every control inside the window. A binding without a `#` source then refers to a property of the view model: `{Binding Name}`.

Two more lines belong in every window that uses a view model:

```
<Window ...
    xmlns:vm="using:HelloMvvm"
    x:DataType="vm:MainWindowViewModel">
```

```

<Design.DataContext>
  <vm:MainWindowViewModel/>
</Design.DataContext>

```

- `x:DataType` tells the compiler the type of the `DataContext`. The bindings are then **compiled**: a misspelled property name is a build error (AVLN2000), not an empty text box at runtime.
- `Design.DataContext` gives the previewer an instance to show, because it can't run the window's constructor.

Lab 4.1: A first view model

🎯 **Goal.** HelloMvvm with a name input and a greeting, bound to a view model.

1. Create the project as in Lab 1.1, named `HelloMvvm`.
2. Add a class `MainWindowViewModel` (file `MainWindowViewModel.cs`) with a `string` property `Name`, initially `""`, and this read-only property:

```

public string Greeting => string.IsNullOrEmpty(Name) ? "Hello, stranger!" :
    ↪ $"Hello, {Name}!";

```

3. In `MainWindow.axaml`, add `xmlns:vm` and `x:DataType` to the `<Window>` start tag as shown above, set `Title="Hello MVVM"` `Width="520"` `Height="560"`, and add the `Design.DataContext` block. Then replace the text `Welcome to Avalonia!` with:

```

<Grid RowDefinitions="*,Auto" Margin="32">
  <StackPanel Grid.Row="0" Spacing="12">
    <TextBox PlaceholderText="Enter your name" Text="{Binding Name}"/>
    <TextBlock Text="{Binding Greeting}" FontSize="24"/>
  </StackPanel>
</Grid>

```

4. In `MainWindow.axaml.cs`, set the `DataContext` in the constructor as shown above.

✅ **Check.** The window shows *Hello, stranger!*. Now type your name: the greeting does **not** change. That is not a mistake in your code. The next chapter explains why.

💡 **Try it.** Misspell `Greeting` in the XAML and build.

5 Change Notification

At the end of Lab 4.1, the greeting ignored your typing. This chapter explains why, fixes it with the Community-Toolkit.Mvvm package, and then lets the user change the view model through two-way bindings.

5.1 Why the greeting doesn't change

Typing in the text box does set `Name`: the binding writes every keystroke into the view model. But nobody tells the `TextBlock` that `Greeting` has changed. The binding read it once, when the `DataContext` was set, and has no reason to read it again.

A view model announces changes with the interface `INotifyPropertyChanged`, which consists of a single event. Every binding subscribes to it, and re-reads its property when the event names it. Written by hand, every property needs this:

```
public class MainWindowViewModel : INotifyPropertyChanged
{
    private string _name = "";

    public string Name
    {
        get => _name;
        set
        {
            if (value == _name) return;
            _name = value;
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nameof(Name)));
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nameof(Greeting)));
        }
    }

    public event PropertyChangedEventHandler? PropertyChanged;
}
```

5.2 CommunityToolkit.Mvvm

The NuGet package **CommunityToolkit.Mvvm** writes this code for you, with a *source generator* that runs at build time:

```
public partial class MainWindowViewModel : ObservableObject
{
    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(Greeting))]
    private string _name = "";
}
```

- `ObservableObject` implements `INotifyPropertyChanged`.
- `[ObservableProperty]` on the private field `_name` generates the public property `Name`, including the notification. You write the field, but use the property.
- `[NotifyPropertyChangedFor(nameof(Greeting))]` additionally announces a change of `Greeting` whenever `Name` changes.
- The class must be **partial**: the generator writes the other half, like the XAML compiler does for a window. In Rider, *Go to Declaration* on `Name` shows the generated code.

To run your own code when a property changes, add a method with a special name; the generator calls it after every change:

```
partial void OnNameChanged(string value) => Console.WriteLine($"Name is now {value}");
```

The toolkit also allows `[ObservableProperty]` on a property instead of a field (`public partial string Name { get; set; }`). You will see that form in later exercises; this book uses the field form.

Lab 5.1: Change notification

🎯 **Goal.** The greeting follows the name as you type.

1. Add the package:

```
dotnet add package CommunityToolkit.Mvvm --version 8.4.0
```

2. Turn `MainWindowViewModel` into the toolkit version from the section above: make it **partial**, derive it from `ObservableObject` (namespace `CommunityToolkit.Mvvm.ComponentModel`), and replace the `Name` property by the field `_name` with its two attributes. Keep `Greeting` as it is.

✅ **Check.** The greeting changes with every letter you type, and shows *Hello, stranger!* again when you empty the text box.

5.3 Two-way binding

Input controls bind `TwoWay` by default, so the user can change any view model property directly: the text of a `TextBox`, the value of a `Slider`, the state of a `CheckBox`. The same property can be bound to several controls at once, and they all stay in sync.

A `bool` property can also switch a style class, as in the chapter *Styles*:

```
<TextBlock Text="{Binding Greeting}" Classes.bold="{Binding IsGreetingBold}"/>
```

Lab 5.2: Two-way binding

🎯 **Goal.** A slider sets the size of the greeting, a check box makes it bold.

1. In the view model, add:

```
[ObservableProperty]  
private double _greetingFontSize = 24;
```

2. In `MainWindow.axaml`, bind the greeting's font size with `FontSize="{Binding GreetingFontSize}"` (instead of 24), and add below it:

```
<Slider Minimum="16" Maximum="48" Value="{Binding GreetingFontSize}"/>
```

3. On your own: add a `bool` property `IsGreetingBold`, a `CheckBox` *Bold* bound to it, and bind the class `bold` of the greeting to it. Define the style `TextBlock.bold` in `<Window.Styles>`.

✅ **Check.** Dragging the slider resizes the greeting, the check box switches bold on and off. There is still no event handler in your code.

5.4 Try it yourself

1. Add a *Shout* check box that shows the greeting in capital letters.

6 Commands

So far the user changes the view model by typing, dragging and checking. Buttons need another mechanism: in a view model world they don't call Click handlers, they execute **commands** of the view model.

6.1 Commands

[RelayCommand] on a method generates a command property with the method name plus Command:

```
[RelayCommand]
private void IncrementCounter() => Counter++;

<Button Content="Increment" Command="{Binding IncrementCounterCommand}"/>
```

6.2 Enabling and disabling a command

A command can decide whether it may run. The button is disabled while it may not:

```
[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof(ResetCounterCommand))]
private int _counter;

[RelayCommand(CanExecute = nameof(CanResetCounter))]
private void ResetCounter() => Counter = 0;

private bool CanResetCounter() => Counter > 0;
```

[NotifyCanExecuteChangedFor] tells the command to ask CanResetCounter again whenever Counter changes.

6.3 Command parameters

A command method can also take a parameter, which the button passes with CommandParameter:

```
[RelayCommand]
private void IncrementBy(int amount) => Counter += amount;

<Button Content="+5" Command="{Binding IncrementByCommand}">
    <Button.CommandParameter>
        <x:Int32>5</x:Int32>
    </Button.CommandParameter>
</Button>
```

Watch out: CommandParameter="5" looks the same, but passes the **string** "5". The command expects an **int**, so it refuses to run, and the button stays disabled without any error message. <x:Int32> creates a real number.

Lab 6.1: Commands

🎯 **Goal.** A counter with *Increment*, *+5* and *Reset*, where *Reset* is disabled at 0.

1. In the view model, add an observable `int` field `_counter` and the `IncrementCounter` command from the section *Commands*.
2. In `MainWindow.axaml`, at the end of the upper `StackPanel`, add:

```
<TextBlock Text="{Binding Counter, StringFormat='Counter: {0}'}"/>
<StackPanel Orientation="Horizontal" Spacing="12">
    <Button Content="Increment" Command="{Binding IncrementCounterCommand}"/>
</StackPanel>
```

`StringFormat` puts the value into a text; `{0}` stands for the value.

3. Add the *+5* button and its command from the section above.
4. On your own: add a *Reset* button and a `ResetCounter` command that is only enabled while the counter is above 0.

✔ **Check.** *Reset* is disabled at the start. *Increment* and *+5* count up and enable it; *Reset* sets the counter to 0 and is disabled again.

💡 **Try it.** Replace the `<Button.CommandParameter>` element by `CommandParameter="5"`.

6.4 Try it yourself

1. Add a *-5* button using the same `IncrementByCommand`.

7 The MVVM Pattern

HelloMvvm now has its state and logic in a view model, bound to the view without a single event handler. This chapter draws the line between view model and view, finishes HelloMvvm, and compares MVVM with the MVC pattern you know from web development.

7.1 What stays in the code-behind

MVVM doesn't mean that every button must be a command. Showing a dialog or closing the window needs the window itself, which the view model deliberately doesn't know. These actions stay in the code-behind. The code-behind may read the view model when it needs data from it:

```
var viewModel = (MainWindowViewModel)DataContext!;
```

The rule of thumb: **state and logic** go into the view model, things that only concern the **window** stay in the view.

Lab 7.1: Dialogs in the code-behind

🎯 **Goal.** *Greet* shows the current greeting in a dialog, *Close* closes the window.

1. Add the package `MessageBox.Avalonia` as in Lab 1.4.
2. In `MainWindow.axaml`, add the lower row of the grid:

```
<StackPanel Grid.Row="1" Orientation="Horizontal" Spacing="12"
    HorizontalAlignment="Center">
    <Button Content="Greet" Classes="accent" Click="OnGreetButtonClicked"/>
</StackPanel>
```

3. In `MainWindow.axaml.cs`, add the using lines for `Avalonia.Interactivity` and `MessageBox.Avalonia`, and the handler:

```
private async void OnGreetButtonClicked(object? sender, RoutedEventArgs e)
{
    var viewModel = (MainWindowViewModel)DataContext!;
    var box = MessageBoxManager.GetMessageBoxStandard("Greeting", viewModel.Greeting);
    await box.ShowAsync();
}
```

4. On your own: a *Close* button next to *Greet*, as in Lab 1.3.

✅ **Check.** *Greet* shows the greeting for the name you typed. *Close* ends the app.

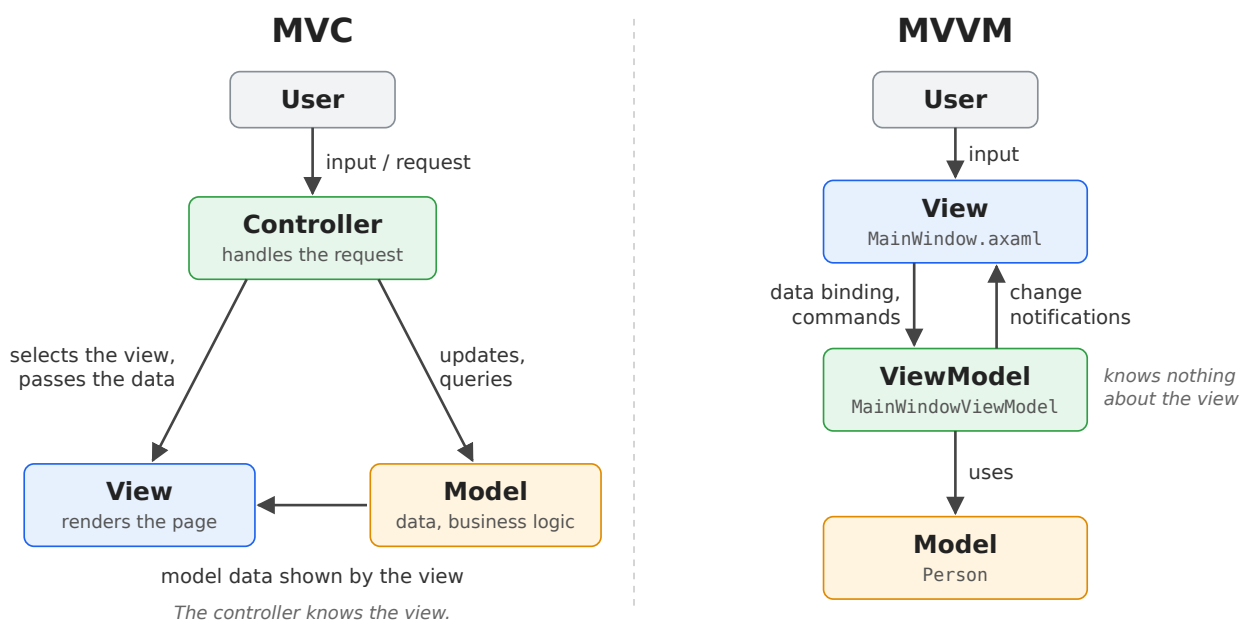
Hello, Ada!

☒ Bold

Counter: 6

7.2 MVVM compared with MVC

From web development you know **MVC** (Model-View-Controller), the pattern behind ASP.NET Core MVC, Spring MVC or Ruby on Rails. Both patterns separate the data (model) from its presentation (view). They differ in what sits between the two, and in how the view is kept up to date.



MVC. Every input, in the web a request, goes to a **controller**. It updates or queries the model, **selects a view** and passes it the data to show. The view renders it, in the web as a new HTML page. The controller knows the view; the view is output only.

MVVM. The input goes to the **view**: the controls themselves. They pass it on to the view model through data binding and commands. The view model holds the state and **knows nothing about the view**: it only announces that something has changed, and the bindings update the controls. Instead of rendering a new page per request, the view stays open and follows the view model continuously.

	MVC	MVVM
Receives the input	Controller	View; bindings and commands forward it
Who knows the view	The controller selects and fills it	Nobody; the view binds to the view model
Updating the display	The controller renders a view, in the web a new page per request	Bindings react to change notifications, continuously
State	Mostly per request, the server stays stateless	Long-lived, in the view model
Glue code between view and logic	Written in the controller	Provided by the data binding framework
Testing without a UI	Controllers, with request and response objects	The view model is a plain class: direct unit tests
Typical for	Server-side web applications	Desktop and mobile UIs: Avalonia, WPF, .NET MAUI

Front-end frameworks such as Angular and Vue are closer to MVVM than to MVC: a component class holds the state, and its template binds to it.

8 Lists and Data Templates

A view model often holds not one object but many. This chapter starts the second lab project, **DatabindingDemo**: a list of people, bound to a `ListBox`, and a data template that decides how each person is shown.

8.1 Lists

To show a list, bind the `ItemsSource` of a `ListBox` to a collection. When items are added or removed later, the list must be told, just like a single property. That's what `ObservableCollection<T>` does: it announces every change of the collection. Changes *inside* an item, like a new name, are announced by the item itself, so the item class is an `ObservableObject`, too.

```
public partial class Person : ObservableObject
{
    [ObservableProperty]
    private string _firstName = "";

    [ObservableProperty]
    private string _lastName = "";

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(IsFullOfAge))]
    private DateTime _birthDate;

    public bool IsFullOfAge => DateTime.Today.AddYears(-18) >= BirthDate;
}

public static class PersonRepository
{
    public static ObservableCollection<Person> GetDummyPeople() => new()
    {
        new Person { FirstName = "John", LastName = "Doe", BirthDate = new DateTime(1975, 1, 1)
        ↵ },
        new Person { FirstName = "Jane", LastName = "Roe", BirthDate = new DateTime(1980, 3, 1)
        ↵ },
        new Person { FirstName = "Lisa", LastName = "Doe", BirthDate = new DateTime(2015, 1, 1)
        ↵ },
    };
}
```

The view model exposes the collection as a property:

```
public ObservableCollection<Person> People { get; } = PersonRepository.GetDummyPeople();
```

Lab 8.1: A list of people

Goal. DatabindingDemo shows the three people in a list.

1. Create the project as in Lab 1.1, named `DatabindingDemo`, and add the package `CommunityToolkit.Mvvm` as in Lab 5.1.
2. Add the classes `Person` and `PersonRepository` from the section above, each in a file of its own.
3. Add a `MainWindowViewModel`: an `ObservableObject` with the property `People`. Set it as `DataContext` in the constructor of `MainWindow`, and add `xmlns:vm`, `x:DataType` and `Design.DataContext` to `MainWindow.axaml` as in Lab 4.1.
4. Set `Title="People"` `Width="480"` `Height="560"` and replace the text `Welcome to Avalonia!` with:

```
<Grid RowDefinitions="Auto,*" Margin="16">
    <ListBox Grid.Row="0" Height="160" ItemsSource="{Binding People}"/>
</Grid>
```

✔ **Check.** The list has three entries, but each one reads *DatabindingDemo.Person*. The list doesn't know how to show a person, so it calls `ToString()`. The next section fixes that.

8.2 Data templates

A **data template** describes how to show an object of a certain type. For the items of a list, it is the `ItemTemplate`:

```
<ListBox ItemsSource="{Binding People}">
  <ListBox.ItemTemplate>
    <DataTemplate x:DataType="vm:Person">
      <StackPanel Orientation="Horizontal" Spacing="5">
        <TextBlock Text="{Binding FirstName}"/>
        <TextBlock Text="{Binding LastName}"/>
      </StackPanel>
    </DataTemplate>
  </ListBox.ItemTemplate>
</ListBox>
```

Inside the template, the `DataContext` is the item, so `{Binding FirstName}` is the first name of this person. `x:DataType` makes these bindings compiled, too.

`StringFormat` formats a value with a .NET format string:

```
<TextBlock Text="{Binding BirthDate, StringFormat='{0:dd.MM.yyyy}'}"/>
```

The `{}` at the start is needed because the format itself starts with a `{`; without it, the XAML parser would take `{0:dd.MM.yyyy}` for a markup extension.

A template can also be stored as a resource with an `x:Key` and used in several places, for example to show a single object with a `ContentControl`:

```
<ContentControl Content="{Binding SelectedPerson}"
  ContentTemplate="{StaticResource PersonTemplate}"/>
```

Lab 8.2: A data template

🎯 **Goal.** Each list entry shows first name, last name and birth date.

1. Add the `ItemTemplate` from the section above to the `ListBox`.
2. On your own: add the birth date as a third `TextBlock`, formatted as `dd.MM.yyyy`.

✔ **Check.** The list shows *John Doe 01.01.1975*, *Jane Roe 01.03.1980* and *Lisa Doe 01.01.2015*.

9 Master/Detail

DatabindingDemo shows a list. This chapter adds a form that edits the selected person, and highlights people by their data, without a single line of refresh code.

9.1 Master/detail

A list and a form that edits the selected entry is called **master/detail**. It needs one more view model property for the selected item, bound to the list's `SelectedItem`:

```
[ObservableProperty]
private Person? _selectedPerson;

<ListBox ItemsSource="{Binding People}" SelectedItem="{Binding SelectedPerson}"/>
```

The form binds through that property with a **path**: `{Binding SelectedPerson.FirstName}`. Because `Person` announces its changes, editing the name in the form updates the list entry at the same time.

As long as nothing is selected, the form should be hidden. A **converter** turns the selected person into a `bool`; Avalonia has a ready-made one for this check:

```
<StackPanel IsVisible="{Binding SelectedPerson, Converter={x:Static
↪ ObjectConverters.IsNotNull}}">
```

Lab 9.1: Master/detail

🎯 **Goal.** Selecting a person opens a form to edit the name.

1. Add the `SelectedPerson` property from above to the view model, and bind the list's `SelectedItem` to it.
2. Below the `ListBox`, add the detail form:

```
<StackPanel Grid.Row="1" Margin="0,16,0,0" Spacing="4"
    IsVisible="{Binding SelectedPerson, Converter={x:Static
        ↪ ObjectConverters.IsNotNull}}">
    <TextBlock Text="First name"/>
    <TextBox Text="{Binding SelectedPerson.FirstName}"/>
</StackPanel>
```

3. On your own: add a label and a text box for the last name.

✅ **Check.** The form is hidden until you select a person. Changing the first name in the form changes the list entry with every keystroke.

9.2 Styling items by their data

The chapter *Styles* switched a style class with a check box. The same works with a property of the data, inside a data template. The `!` in front of a path negates a `bool`:

```
<StackPanel Orientation="Horizontal" Spacing="5" Classes.underage="{Binding !IsFullOfAge}">

<Style Selector="StackPanel.underage">
    <Setter Property="Background" Value="LightCoral"/>
</Style>
```

Now everything fits together. A command that changes a person's birth date sets the property `BirthDate`. The generated setter announces `BirthDate` and, because of `[NotifyPropertyChangedFor]`, `IsFullOfAge`. The formatted birth date and the highlighted row update on their own, without knowing about each other and without any refresh code.

Lab 9.2: Data-driven highlighting

🎯 **Goal.** People under 18 are highlighted, and a button makes the selected person one year older.

1. Add `Classes.underage` to the template's `StackPanel` and the `StackPanel.underage` style to

<Window.Styles>, as shown above.

2. On your own: add a command `MakeOneYearOlder` that moves the selected person's birth date one year back (`BirthDate.AddYears(-1)`). It may only run while a person is selected. Add a button *Make one year older* at the end of the detail form.

✓ **Check.** Lisa's row is red. Select her and click *Make one year older* a few times: her birth date moves back, and when she turns 18 the red disappears.

The screenshot shows a WPF application window. At the top, there is a list of three people, each on a new line: "John Doe 01.01.1975", "Jane Roe 01.03.1980", and "Lisa Doe 01.01.2015". The row for "Lisa Doe 01.01.2015" is highlighted with a red background. Below the list, there is a detail form. It has two text boxes: "First name" with the value "Lisa" and "Last name" with the value "Doe". Below these text boxes is a button labeled "Make one year older".

9.3 Try it yourself

1. Add an *Add person* button that adds a new, empty person to `People` and selects it, and a *Remove* button for the selected person.
2. Replace the detail form with a `ContentControl` and a keyed `DataTemplate`, as shown in the chapter *Lists and Data Templates*. What happens to the form when nothing is selected?

Part 3

Multithreading

10 CPU-bound vs. I/O-bound

A desktop application may use every core of the machine, and a calculation that runs for seconds is worth spreading across all of them. This part is the theory for that: it has no labs, and the -calculator exercise puts it into practice. Its chapters follow three questions, in the order you should ask them:

1. **What is my code waiting for?** CPU-bound vs. I/O-bound (this chapter)
2. **How do I run it in parallel?** *Threads and Tasks*
3. **What breaks when I do?** *Race Conditions*, and *Protecting Critical Sections*

The last chapter, *The UI Thread*, shows the extra rule a UI application adds.

10.1 What is my code waiting for?

Before reaching for a thread, work out what the code is actually waiting for. This single question decides which tool is correct — and picking the wrong one usually makes things *worse*, not just no better.

	CPU-bound	I/O-bound
The code is waiting for	a processor core	a device (disk, network, database)
While waiting, the CPU is	busy at 100%	idle
Typical examples	Monte Carlo simulation, image filters, encryption, sorting a huge list, parsing	HTTP requests, file reads, SQL queries, waiting for user input
Gets faster with more threads	yes , up to your core count	no
The right tool	<code>Task.Run</code> / threads / SIMD	<code>async/await</code>
What you gain	throughput — the work finishes sooner	scalability — you can serve far more work at once

The single most important thing to understand before any of the rest: **`async/await` does not create threads**. It is a mechanism for *giving a thread back* while you wait for something that isn't the CPU. The sections below show what that means in each of the two cases.

10.2 The Task Parallel Library

TPL stands for **Task Parallel Library** — the `System.Threading.Tasks` namespace, added in .NET 4 and the standard way to do concurrency in .NET ever since. `async/await`, `Task.Run`, `Task.WhenAll`, `Parallel.For` and `PLINQ` are all part of it.

The shift it introduced is one of vocabulary, and it is worth stating plainly:

With threads you describe **workers** — “*create a worker, start it, wait for it to finish.*” With the TPL you describe **work** — “*here is a job; run it and give me the result.*” Which thread runs that job, and when, becomes the library's problem rather than yours.

The TPL runs work on the **thread pool**: a set of reusable worker threads that it hands work items to. The chapter *Threads and Tasks* compares the two models in detail.

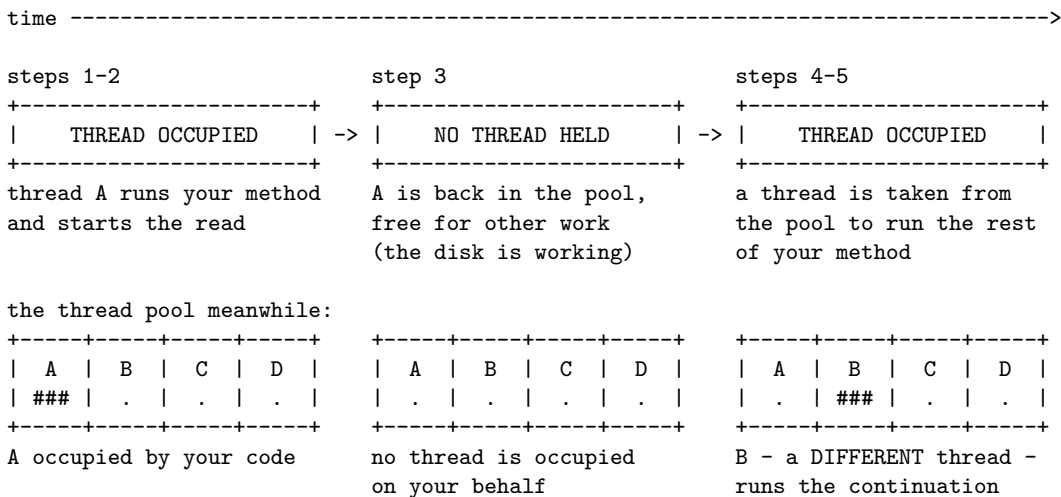
10.3 I/O-bound work: waiting without a thread

So what actually happens when you `await` an I/O call? Follow one line through, step by step:

```
var text = await File.ReadAllTextAsync(path);
Console.WriteLine(text.Length);           // the "continuation"
```

1. Your method asks the operating system to start the read and **returns immediately** — it does not wait. The caller gets back an unfinished **Task**.
2. The thread that was running your method is now free. It goes back to the **thread pool** and does something else.
3. The disk works. **No thread is involved at all** during this time — the operation is in the hands of the hardware and the OS.
4. When the data is there, the OS notifies .NET, which schedules the **rest of your method** — the code after the **await** — as a new work item.
5. A thread picks that work item up and runs `Console.WriteLine(text.Length)`. It may well be a *different* thread than the one from step 1.

Drawn on a time axis, with a four-thread pool underneath:



The middle phase is not drawn to scale: reading a file takes milliseconds and a network call tens of milliseconds, while the two occupied phases are a few microseconds each. In other words, **the overwhelming majority of the elapsed time is spent in the middle box, where your code holds no thread at all**. Had you used the blocking `File.ReadAllText` instead, thread A would have stayed occupied — and useless — across all three boxes.

The key point: a thread is held only while your code is actually executing, never while it is waiting for a device. That is what lets a web server handle tens of thousands of simultaneous requests on a handful of threads — nearly all of them are waiting, and waiting is free.

10.4 CPU-bound work: moving it off the caller

Now the other case. This looks almost identical at the call site:

```
var result = await Task.Run(() => Calculate(1_000_000_000));
```

but the middle box of the diagram above simply does not exist. There is no device to wait for — a core has to execute those billion iterations — so a thread stays **occupied for the entire duration**. Nothing is given back. That means `Task.Run` here **does not make the calculation faster**. What it does is move the work **off the calling thread**. That matters enormously when the caller is the UI thread: without it the window freezes for the whole calculation, because the thread that should be redrawing the window is busy doing arithmetic instead. The **await** keeps the UI responsive; it does not shorten the computation by a single millisecond.

The actual speed-up has to come from somewhere else — **splitting the work across several cores**, which is what the next chapters are about.

	I/O-bound <code>await</code>	CPU-bound <code>Task.Run</code>
Thread held while working	no	yes , the whole time

	I/O-bound <code>await</code>	CPU-bound <code>Task.Run</code>
What <code>await</code> buys you	scalability — thousands of waits, few threads	responsiveness — the caller stays free
How it gets faster	it doesn't need to; waiting is free	more cores, or SIMD

10.5 Two mistakes worth naming

Fake async — wrapping blocking I/O in `Task.Run`:

```
// BAD: burns a pool thread that just sits there waiting for the disk
var text = await Task.Run(() => File.ReadAllText(path));

// GOOD: no thread is held while the disk works
var text = await File.ReadAllTextAsync(path);
```

The first version is not more scalable than the blocking call — it is *less*, because you now block a **pool** thread instead of your own, and the pool has better things to do. On a server this is a classic cause of thread-pool starvation.

Blocking on async code — `.Result` or `.Wait()`:

```
var data = FetchAsync().Result; // BAD: blocks, and can deadlock outright
```

This throws away the entire benefit and, in a context with a synchronization context (older UI frameworks, ASP.NET), can deadlock: the continuation needs the thread you are blocking. Rule of thumb: **async all the way up**.

10.6 Summary

`async/await` makes code **wait without blocking**. Threads make code **run in parallel**. Only CPU-bound work benefits from more threads, and only I/O-bound work benefits from **async**.

A simulation that does nothing but arithmetic — no disk, no network — is a textbook CPU-bound problem. That is precisely why multithreading pays off for it, and why **async** alone would achieve nothing.

11 Threads and Tasks

There are two ways to run CPU-bound work in parallel: create threads yourself, or hand tasks to the TPL. This chapter writes the same job both ways, and then compares the two models.

11.1 The same job, written twice

Which delegate each one accepts decides how your worker method has to be shaped, so start there:

Started with	Delegate it accepts	Can hand a result back?
<code>new Thread(...)</code>	<code>ThreadStart</code> — <code>void ()</code>	no
<code>new Thread(...)</code>	<code>ParameterizedThreadStart</code> — <code>void (object?)</code>	no, and the argument is an untyped object? you must cast
<code>Task.Run(...)</code>	<code>Action</code> — <code>void ()</code>	no — gives a plain <code>Task</code>
<code>Task.Run(...)</code>	<code>Func<TResult></code> — <code>TResult ()</code>	yes — gives a <code>Task<TResult></code>

Both `Thread` delegates return `void`. That is not a detail — it is *the* reason the manual version below needs an array: a thread has nowhere to put a return value, so a result has to escape through a captured variable instead.

The worker method itself is the same ordinary method in both cases — it takes a parameter and returns a result:

```
double Calculate(long amount) { /* ... some CPU-bound work ... */ }
```

Run four of them and average the results — first manually:

```
// MANUAL: ThreadStart is `void ()`. Calculate's return value cannot travel out through
// the delegate, so each thread assigns it to its own slot of a shared array instead.
const long amount = 1_000_000;
var threads = new Thread[4];
var results = new double[4]; // the only way out for the results

for (var i = 0; i < 4; i++)
{
    var index = i; // capture a COPY - all four lambdas
                  // would otherwise share one `i`
    threads[i] = new Thread(() => results[index] = Calculate(amount));
    threads[i].Start(); // one real OS thread each
}

foreach (var thread in threads) thread.Join(); // blocks here until all finish
var average = results.Average();
```

and then with the TPL:

```
// TPL: Task.Run accepts a Func<double>, so Calculate's return value flows straight out
// as Task<double> - no array, no captured slot, nothing to index.
const long amount = 1_000_000;
var tasks = new Task<double>[4];

for (var i = 0; i < 4; i++)
{
    tasks[i] = Task.Run(() => Calculate(amount)); // Func<double> -> Task<double>
}

double[] results = await Task.WhenAll(tasks); // hands back a double[] directly
var average = results.Average();
```

Three things disappeared in the second version, and they are the TPL's whole value proposition:

- **No results array and no index to capture** — because `Func<TResult>` can return a value, each task carries its own result out, so there is no shared slot to write into and no loop-variable trap to fall into.
- **No Join loop, and no blocking** — `await` releases the calling thread instead of parking it, exactly as with I/O-bound work in the chapter *CPU-bound vs. I/O-bound*.

- **No thread management** — nothing is created or destroyed. `Task.Run` puts a work item on the thread pool, and a pool thread that already exists picks it up.

Careful with captured variables: a lambda captures the *variable*, not its value. That is why `index` above is a fresh variable declared inside the loop — capture `i` directly and all four threads may end up seeing the same, final value.

You also get three things for free that the manual version would each need hand-writing:

- **Exceptions propagated to the caller** — the `Task` captures the exception and rethrows it at the `await`, instead of an unhandled exception on a thread taking the whole process down with it.
- **Cooperative cancellation** — a `CancellationToken` handed to the work, instead of a stop flag you have to invent and wire up yourself.
- **Composition** — `WhenAll`, `WhenAny` and continuations, instead of `Joining` threads one at a time and hoping you got the order right.

11.2 Thread: the explicit model

Every `new Thread` is a genuine operating-system thread with its own stack (1 MB by default on Windows). Creating one costs on the order of a hundred microseconds and some kernel bookkeeping — irrelevant if you create eight, ruinous if you create one per work item.

`Thread` gives you low-level control and very little else. It cannot return a value, so results have to be written into a captured variable or array. An unhandled exception inside a thread **terminates the whole process**. There is no built-in cancellation and no way to compose several threads other than `Joining` them one by one.

11.3 Task: the TPL model

A `Task` is a *unit of work*, not a thread. By default it is queued onto the **thread pool**, which maintains a set of reusable worker threads (roughly one per logical processor to start with) and hands them work from a queue. Nothing is created or destroyed per work item.

On top of that, `Task` provides everything `Thread` lacks: `Task<T>` carries a **return value**; exceptions are **captured** and rethrown at the `await`; `CancellationToken` gives cooperative **cancellation**; and `WhenAll`/`WhenAny`/continuations let you **compose** work.

11.4 Side by side

	Thread	Task (TPL)
What it is	an OS thread	a unit of work, usually run on a pool thread
Creation cost	high (new stack, kernel object)	low (queued; threads are reused)
Returns a value	no	yes — <code>Task<T></code>
Waiting	<code>Join()</code> (blocks)	<code>await</code> (does not block)
Unhandled exception	kills the process	captured in the <code>Task</code> , rethrown on <code>await</code>
Cancellation	build it yourself	<code>CancellationToken</code>
Composition	manual	<code>WhenAll</code> , <code>WhenAny</code> , continuations
Foreground/background	foreground by default — keeps the process alive	always background
Count you should create	a few, deliberately	as many as the work suggests

Default answer: use `Task`. Reach for `Thread` only when you need something the pool cannot give you — a specific priority, a custom stack size, a COM apartment state, or a genuinely long-lived dedicated loop.

11.5 The thread pool grows slowly

The pool starts with about **one worker thread per logical processor**. If all of them are busy and more work is queued, it does *not* simply add more: it assumes the running items will finish shortly, and adds threads only **one at a time, slowly** — measured on an 8-core machine, roughly one per second. Queue 60 blocking work items and six seconds later 47 of them still have not started.

That policy has a direct consequence for the choice between **Thread** and **Task**:

- **32 Threads** → 32 real threads, all running at once (and fighting over 8 cores).
- **32 Task.Run** → about 8 run; the other 24 **sit in a queue**.

So with tasks, “32 units” does not mean “32 things running at once”. If you benchmark thread counts using `Task.Run`, past your core count you are measuring the pool’s ramp-up policy rather than parallelism.

Worse, long CPU-bound work on pool threads starves **everything else** that needs the pool. A measured example: a UI refresh loop driven by `Task.Delay(50)` should tick 20 times per second, but the timer callback that completes a `Task.Delay` itself needs a pool thread —

Calculation started with	UI frames in 4 seconds
<code>Task.Run</code> (8 CPU-bound tasks on an 8-core machine)	2
<code>Task.Factory.StartNew(..., TaskCreationOptions.LongRunning)</code>	58

The fix is to tell the TPL that this work is not a short job:

```
Task.Factory.StartNew(() => LongCpuWork(), TaskCreationOptions.LongRunning);
```

`LongRunning` gives the task a **dedicated thread** instead of a pool thread, keeping the pool free. You keep the whole `Task` API — return values, `await`, `WhenAll` — while behaving like a manual thread underneath. For CPU-bound work that runs for seconds, this is usually what you want.

11.6 One level up: Parallel and PLINQ

For data-parallel loops you often don’t need to manage tasks at all:

```
Parallel.For(0, workers, i => results[i] = Calculate(amount));

var average = Enumerable.Range(0, workers).AsParallel().Select(_ => Calculate(amount)).Average();
```

These partition the work, pick a degree of parallelism, and join for you. Less control, much less code. They are still built on the thread pool, so the caveat above applies.

12 Race Conditions

Running work in parallel is the easy part. As soon as two threads touch the same data, results can go wrong in ways that are hard to see and harder to reproduce. This chapter shows how, and the next one how to prevent it.

12.1 What a race condition is

A **race condition** is a bug where the result depends on the *relative timing* of two or more threads. The same code, run twice on the same input, can produce different answers.

A race needs **shared mutable state**: two or more threads reaching the same variable, and at least one of them writing. Remove any one of those three — sharing, mutation, or concurrency — and the race is gone.

12.2 Anatomy: the lost update

This innocent line is the classic example:

```
counter += batch;
```

It looks like one operation. At machine level it is **three**:

1. **read** the current value of **counter**
2. **add batch** to it
3. **write** the result back

Nothing prevents another thread from doing its own read *between* your read and your write:

thread A	thread B	counter
read → 5000		5000
	read → 5000	5000
add 400 → 5400		5000
	add 300 → 5300	5000
write 5400		5400
	write 5300	5300 ← A's 400 is lost

Both threads did their job, and one update vanished. This is a **lost update**.

It is not theoretical. Eight threads reporting their progress into one shared counter, 2,000,000 increments in total, ten consecutive runs of the identical program:

run	counted	lost
1	1,889,250	110,750
2	1,987,500	12,500
3	1,976,450	23,550
10	1,997,850	2,150

Wrong in **10 out of 10 runs**, and never by the same amount. **A result that changes between runs of identical code is the signature of a race condition.**

Run it yourself: the console program *CounterThreadSynchronization*, handed out with the - calculator exercise, does exactly this, and then repeats it with each of the fixes described in the chapter *Protecting Critical Sections* — so you can watch the losses appear, disappear, and compare what each mechanism costs.

12.3 Why these bugs are dangerous

Notice what did *not* happen: nothing crashed, and no exception was thrown. In those same runs the threads' actual computed results were all correct, because each thread kept its own partial result in its own storage and shared nothing. Only the shared counter was wrong.

Concurrency bugs routinely look exactly like this — a plausible answer, no error, slightly wrong. You cannot wait for a crash to find them. You have to **reason about which state is shared**.

12.4 Other kinds of races

Check-then-act. The check and the action are separately correct but not atomic:

```
if (!dictionary.ContainsKey(key))    // thread B can insert right here
    dictionary.Add(key, value);      // -> exception, or a lost value
```

Torn reads. A read that catches a value mid-write. On a 32-bit runtime a `long` is written in two 32-bit halves, so a reader can observe half of the old value and half of the new — a number that was never assigned.

Stale reads and reordering. Compiler and CPU may cache a value in a register or reorder instructions. A thread can keep seeing an old value long after another thread changed it. This is what `volatile` addresses — and *only* this.

13 Protecting Critical Sections

A **critical section** is a region of code that must not be executed by more than one thread at a time. The options below go from cheapest and most preferable to most specialised; the table at the end helps to choose.

13.1 Don't share (always try this first)

The best fix is to remove the sharing. Give each worker its own storage and combine at the end:

```
var results = new double[workers];
// each thread writes only results[myIndex] - distinct memory, no synchronization needed
...
var average = results.Average();    // combine on one thread, after all Joins
```

Different array elements are independent locations, so this is safe with no locking at all. Immutable data is likewise safe by construction. **Most concurrency bugs are best fixed by not sharing, not by locking harder.**

13.2 Interlocked: atomic operations on a single variable

```
Interlocked.Add(ref _counter, batch);
Interlocked.Increment(ref _counter);
var old = Interlocked.Exchange(ref _flag, 1);
Interlocked.CompareExchange(ref _value, newValue, expected);
```

These perform read-modify-write as **one indivisible CPU instruction**. No thread can interleave, and no thread ever waits — this is *lock-free*, and the cheapest correct option.

The catch: it needs a **ref** to a **field**, and you cannot pass a property with **ref**. That is why a counter exposed as a property needs an explicit backing field:

```
private long _processed;           // the field Interlocked works on
public long Processed => _processed; // read-only view for the outside world

public void Report(long batch) => Interlocked.Add(ref _processed, batch);
```

Use it when you are guarding **one** variable. It cannot make two updates atomic together.

13.3 lock: mutual exclusion

```
private readonly object _sync = new();

lock (_sync)
{
    _total += batch;    // several statements can be made atomic
    _count++;          // as one unit
}
```

`lock` is syntactic sugar for `Monitor.Enter/Exit` in a `try/finally`. Use it when an **invariant spans more than one variable** — exactly what `Interlocked` cannot do.

Rules:

- Lock on a **private readonly object**. Never `lock(this)` (outside code can lock your instance), never a `string` (interned — shared process-wide), never a `Type`.
- **Keep it short**. Every other thread that arrives has to wait; a long critical section serialises your program and undoes your parallelism.
- **No await inside a lock** — it will not compile, and for good reason: a lock is owned by a *thread*, and the continuation after an `await` may run on a different one.
- Always take multiple locks in the **same order** everywhere. Two threads taking $A \rightarrow B$ and $B \rightarrow A$ is a **deadlock**: each holds what the other needs, and neither ever proceeds.

In .NET 9+, prefer the dedicated `System.Threading.Lock` type over a plain object — it is faster and `lock` (`myLock`) gets compile-time checking. A plain object still works everywhere.

13.4 Concurrent collections

```
var queue = new ConcurrentQueue<Dot>(); // producers Enqueue, consumer TryDequeue
var map = new ConcurrentDictionary<K, V>();
```

Each individual operation is thread-safe, so you do not need your own lock around them. But **compound** operations still are not atomic — if `(!map.ContainsKey(k)) map.TryAdd(k, v)` has the same check-then-act bug as in the chapter *Race Conditions*. Use the purpose-built atomic methods instead: `GetOrAdd`, `AddOrUpdate`, `TryUpdate`.

13.5 SemaphoreSlim: when you need to await inside

```
private readonly SemaphoreSlim _gate = new(1, 1);

await _gate.WaitAsync();
try { await SomethingAsync(); }
finally { _gate.Release(); }
```

The async-friendly alternative to `lock`, and the tool for limiting concurrency to n rather than 1. Always `Release` in a `finally`.

13.6 ReaderWriterLockSlim: many readers, rare writers

Lets any number of readers in simultaneously but excludes them while a writer works. Worth it only when reads dominate heavily and the critical section is genuinely expensive; otherwise `lock` is simpler and often faster.

13.7 What volatile does — and does not — do

`volatile` is the most commonly misapplied keyword in this area.

```
private volatile bool _stopRequested; // GOOD: a flag one thread sets, another polls
private volatile long _counter;       // USELESS: counter += x is still not atomic
```

`volatile` guarantees **visibility and ordering** — you read a fresh value rather than a cached one, and the compiler will not reorder around it. It says **nothing about atomicity**. All three steps of `+=` can still interleave, and updates are still lost. (It also cannot be applied to `long` or `double`, so the second line does not even compile.) For a counter you need `Interlocked`; for a stop flag `volatile` is exactly right.

13.8 Choosing

Situation	Use
Each worker can have its own copy	no synchronization — combine at the end
One counter or flag, simple update	<code>Interlocked</code>
A flag one thread sets, another reads	<code>volatile</code> (or <code>Volatile.Read/Write</code>)
Invariant across several fields	<code>lock</code>
Shared collection	<code>ConcurrentQueue</code> / <code>ConcurrentDictionary</code>
Need to <code>await</code> in the critical section	<code>SemaphoreSlim</code>
Reads vastly outnumber writes	<code>ReaderWriterLockSlim</code>

14 The UI Thread

14.1 Only the UI thread touches the UI

GUI frameworks add a rule the compiler cannot enforce: **only the UI thread may touch UI objects**. This is not a lock you take — it is a hard constraint. Background work that wants to update the screen must hand the data over. In Avalonia, the `Dispatcher` does that:

```
Dispatcher.UIThread.Post(() => ProgressValue = processed);
```

14.2 Don't flood the UI thread

Two things to watch. Marshalling per item floods the UI thread — a worker reporting tens of thousands of progress updates will swamp it — and the blocking variants (`Invoke` rather than `Post`) additionally stall the *worker* on the UI thread, which distorts any timing you try to measure. The pattern that scales is to let workers write to a thread-safe collection or an `Interlocked` counter, and have the UI read it on a timer at a fixed frame rate.

14.3 Further reading

For the whole part, *Multithreading*:

- [Asynchronous programming scenarios](#) — CPU-bound vs. I/O-bound guidance
- [Task Parallel Library](#)
- [Managed threading best practices](#)
- [Interlocked](#) · [lock statement](#) · [Thread-safe collections](#)
- [Cancellation in managed threads](#)